

From RNNs to Attention and Transformers

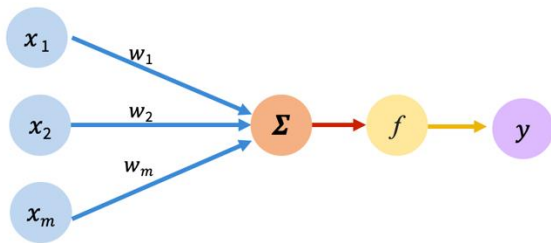
Plan

- Recap on the Perceptron
- Recurrent Neural Networks (RNNs)
- Long-Short Term Memory (LSTMs)
- Attention
- Transformers with self-attention

Recap on Perceptrons and Neural Networks

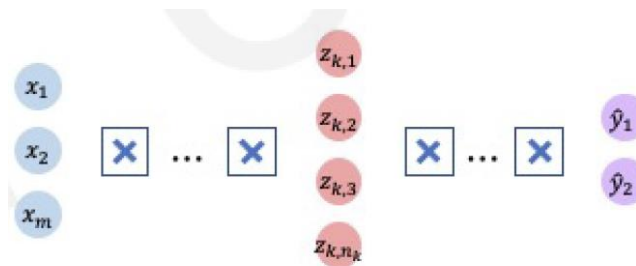
The Perceptron

- Structural building blocks
- Nonlinear activation functions



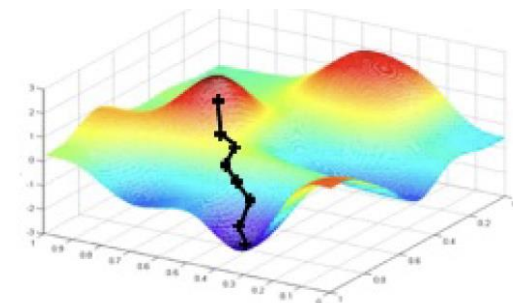
Neural Networks

- Stacking Perceptrons to form neural networks
- Optimization through backpropagation



Training in Practice

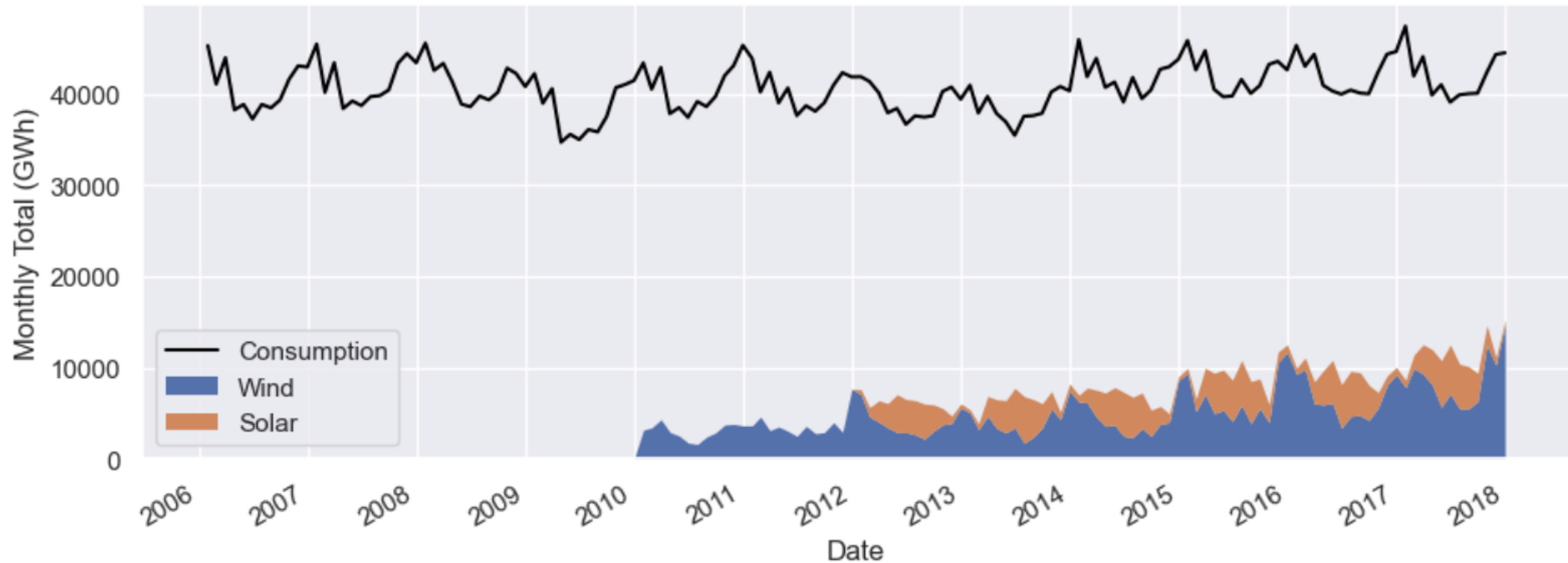
- Adaptive learning
- Batching
- Regularization



Recurrent Neural Networks

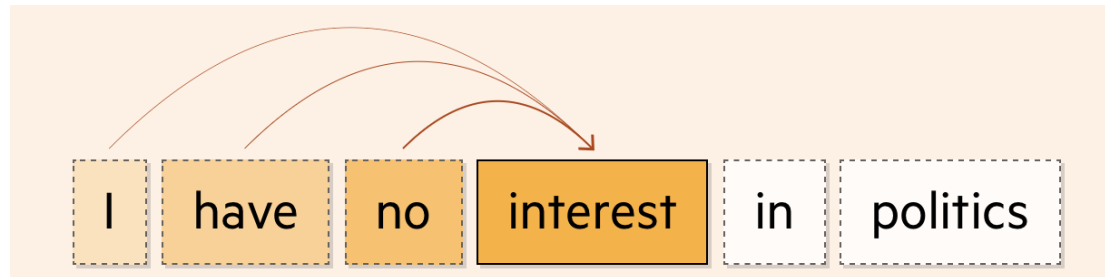
Modelling of sequences

- We've seen time series before:

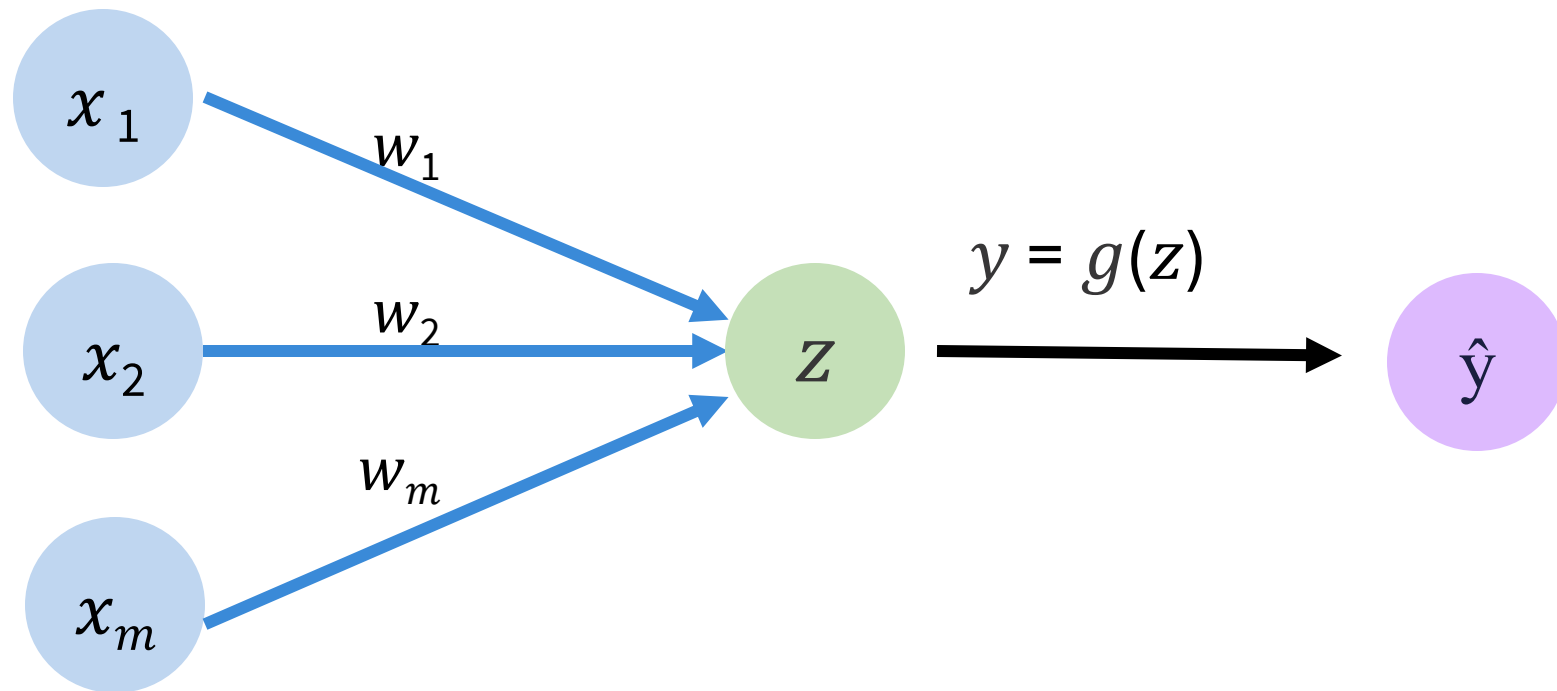


Modelling of sequences

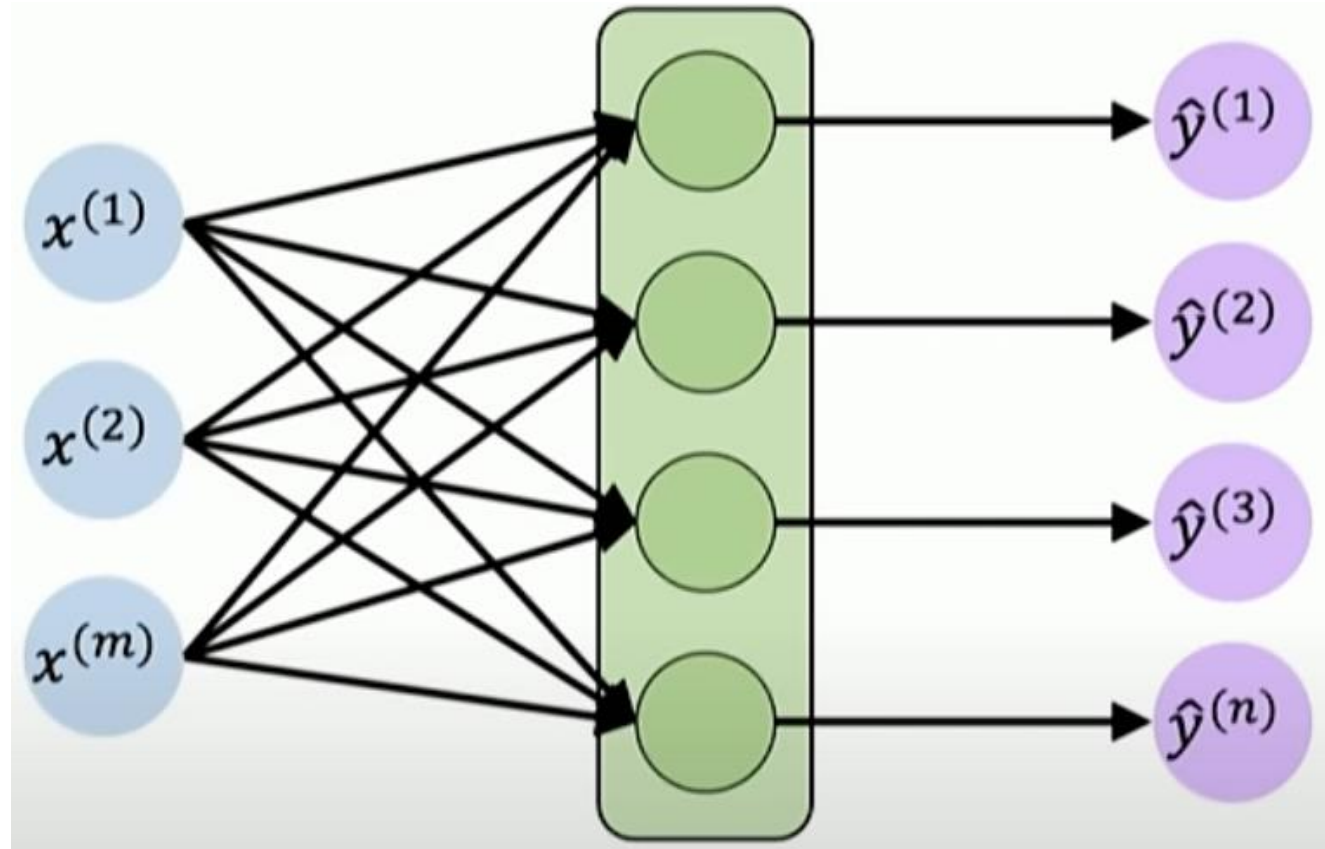
- Text also follows a sequence:
 - “Hello, how are ____”
 - “Grandma, what big teeth you ____”
 - “To Infinity and ____”
- With NLP pre-processing steps, we split these words into tokens.
- Sequences were traditionally modelled with **recurrence** (RNNs)
- Each word is scanned and processed sequentially



The Perceptron Revisited



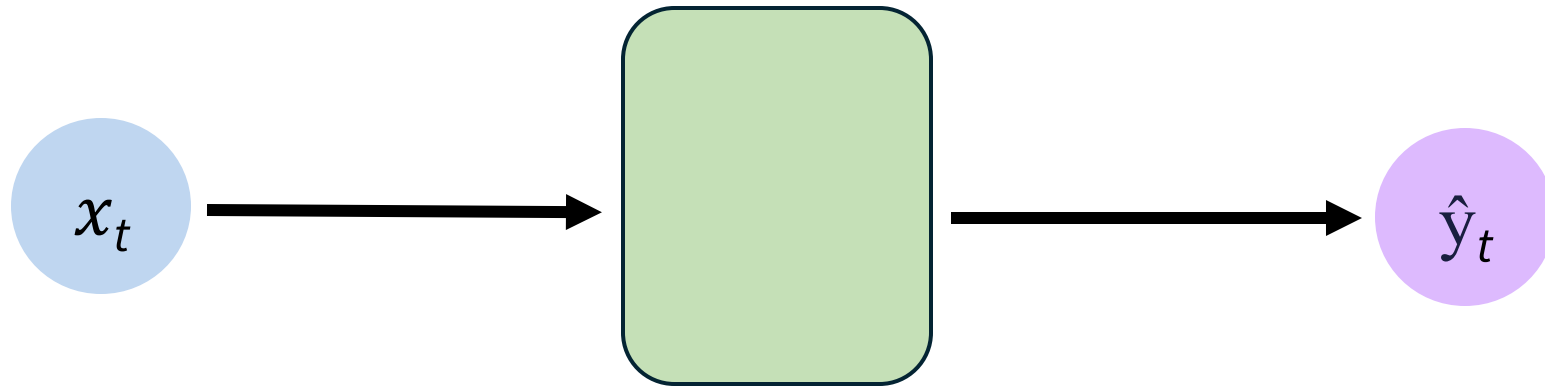
Feed-Forward Networks Revisited



$$\mathbf{x} \in \mathbb{R}^m$$

$$\hat{\mathbf{y}} \in \mathbb{R}^n$$

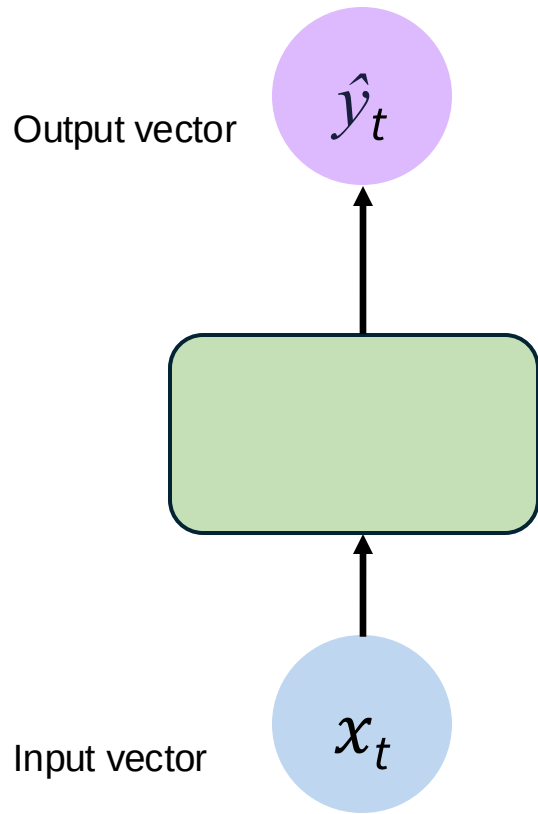
Feed-Forward Networks Revisited



$$\mathbf{x}_t \in \mathbb{R}^m$$

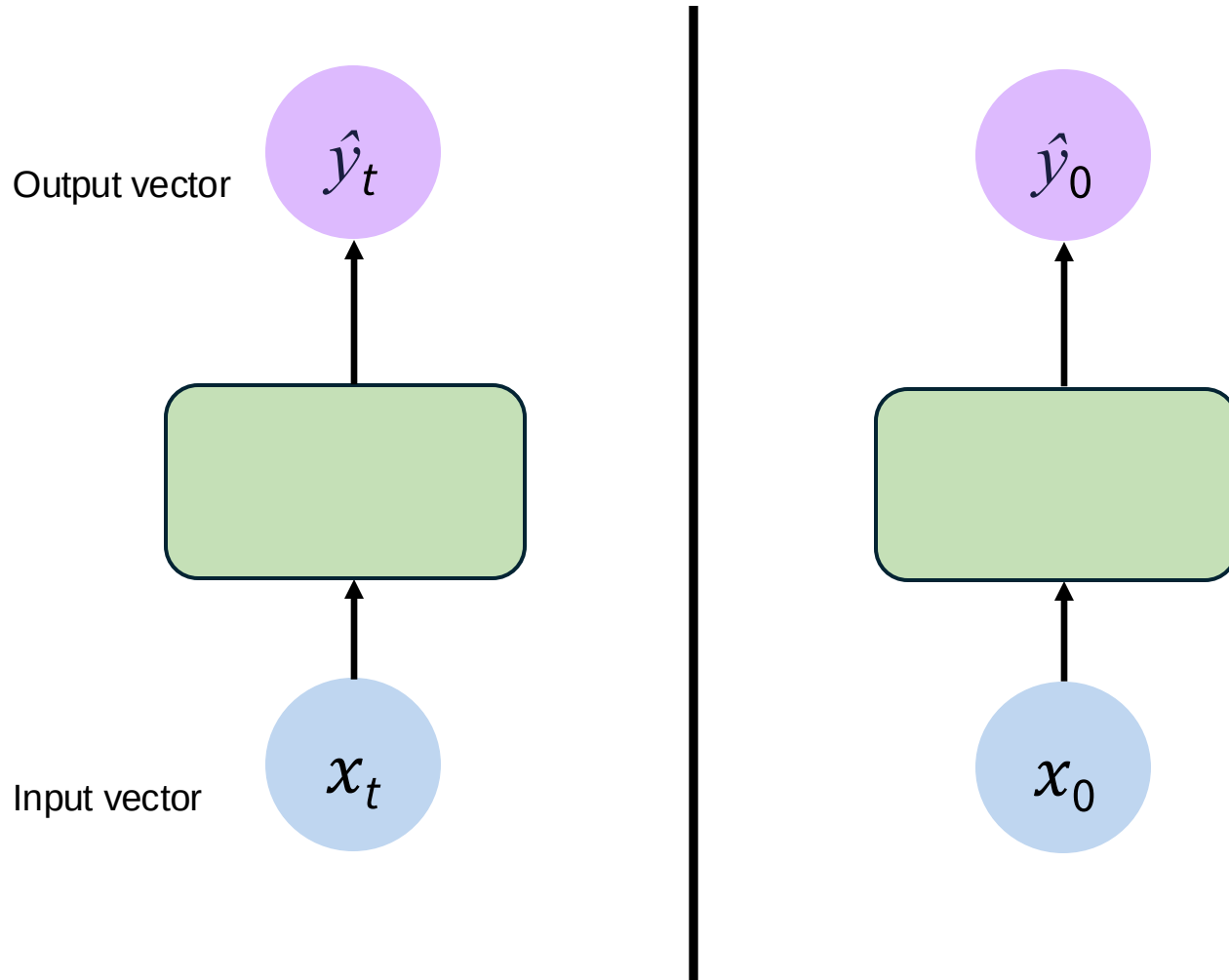
$$\hat{\mathbf{y}}_t \in \mathbb{R}^n$$

Handling Individual Time Steps

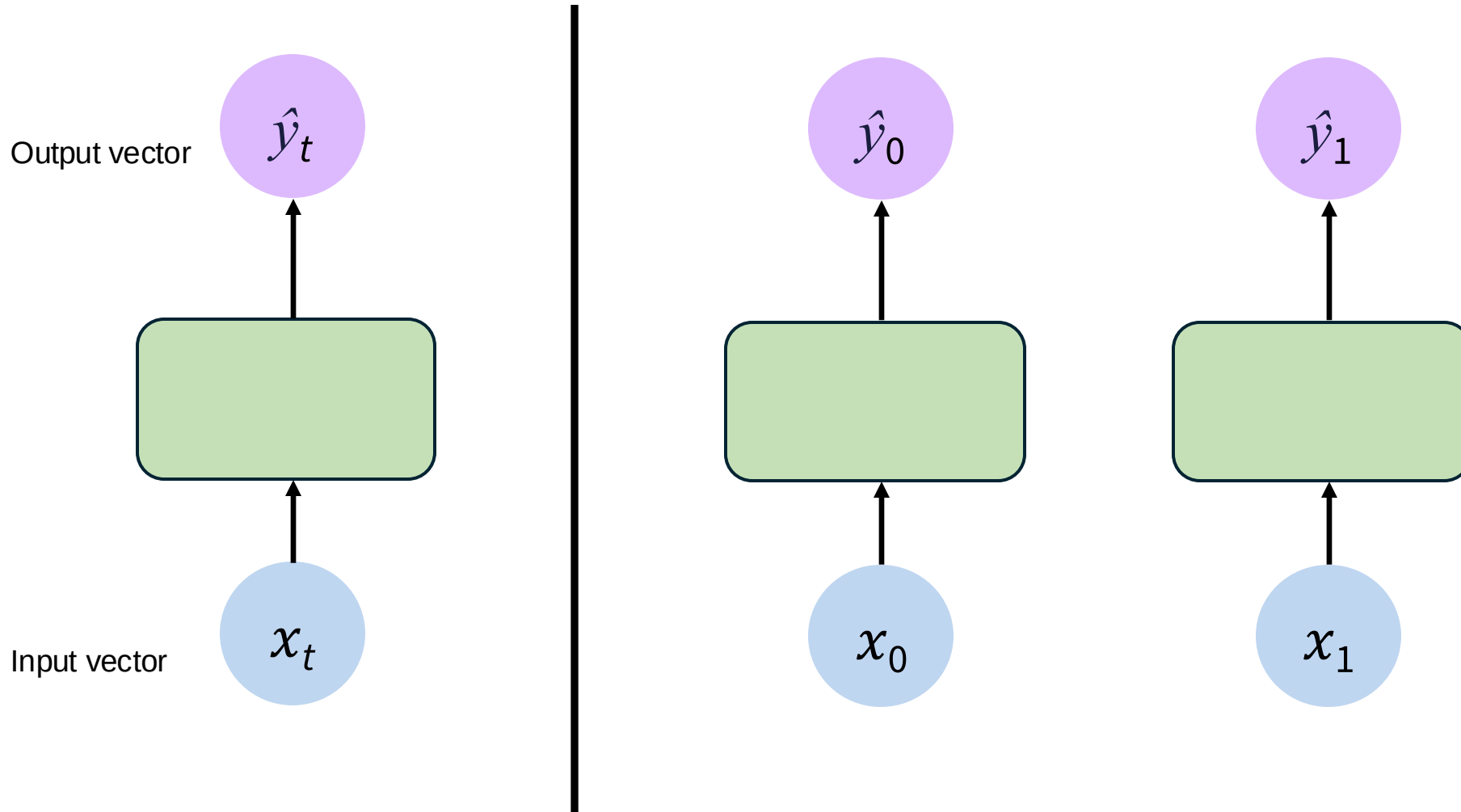


Click through the time steps

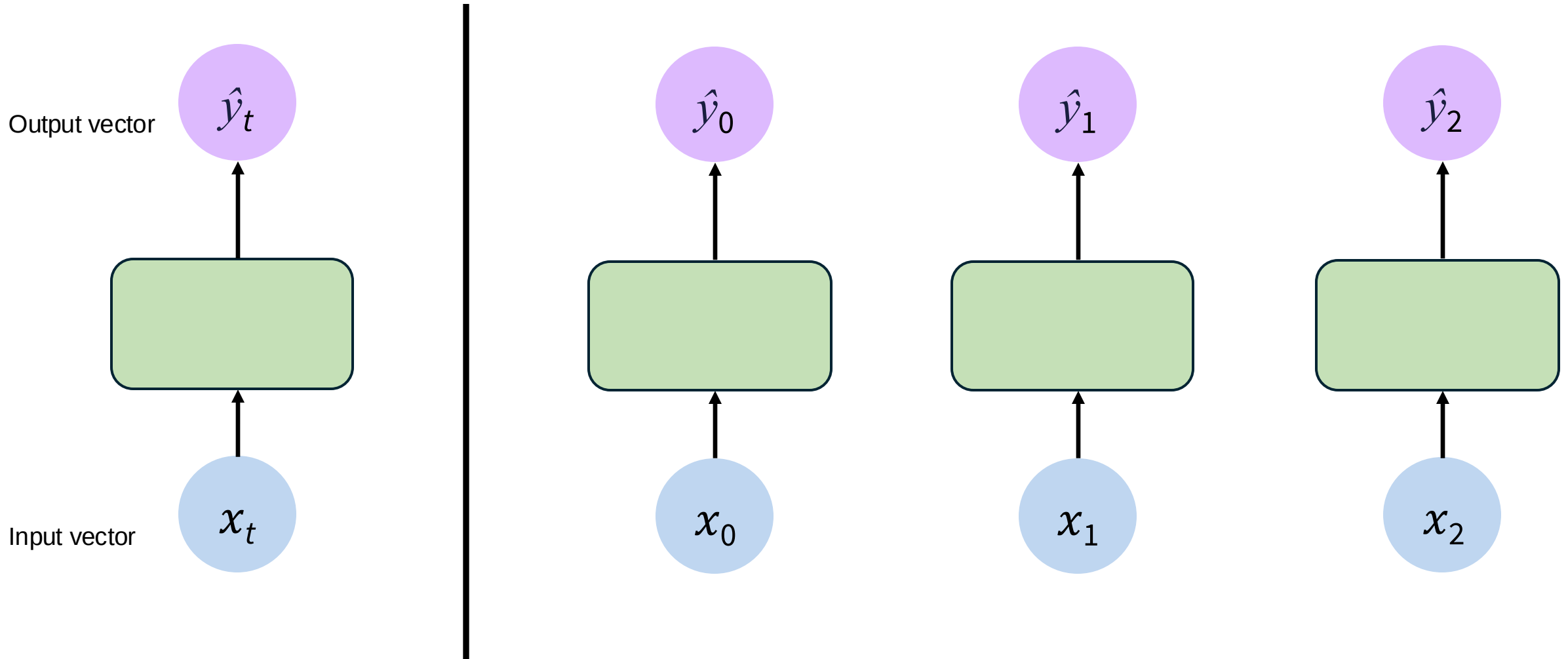
Handling Individual Time Steps



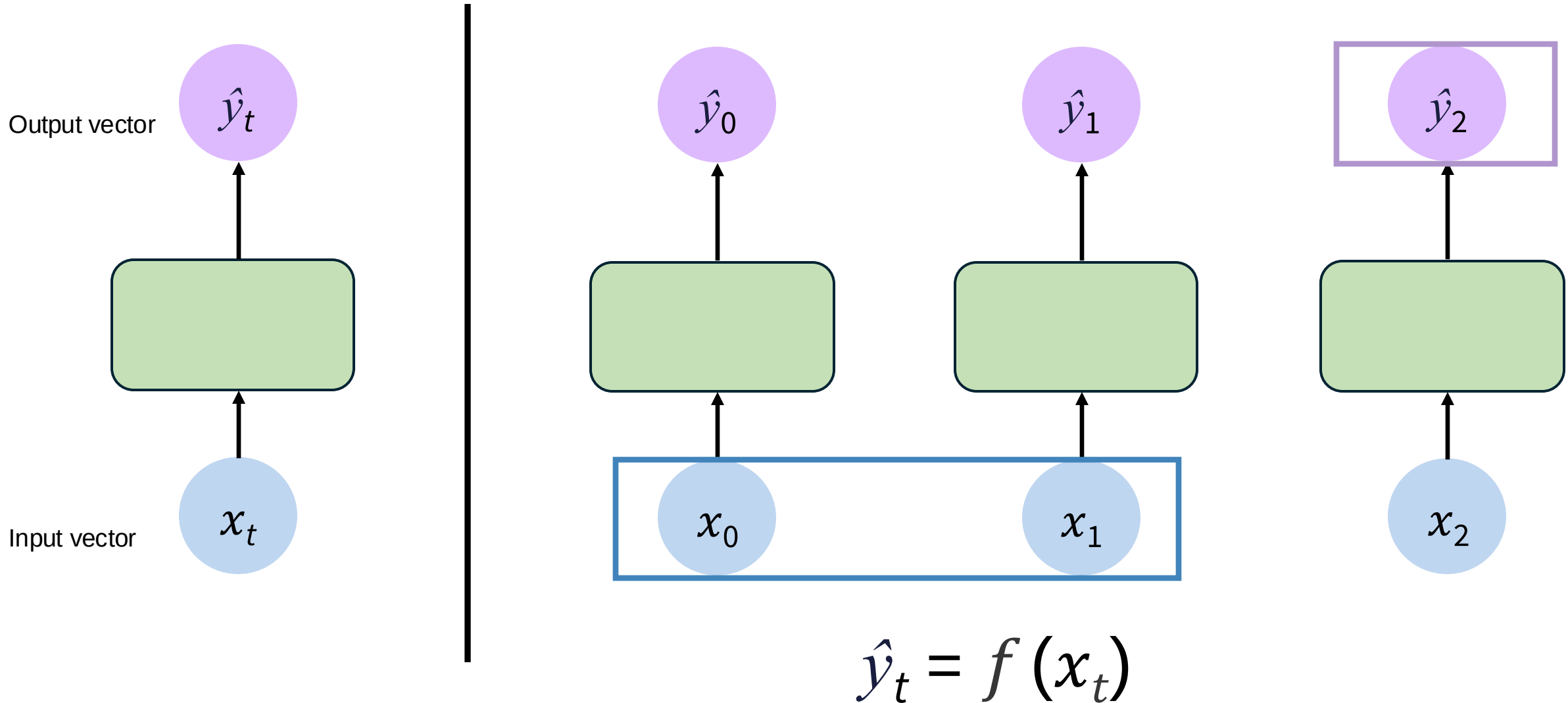
Handling Individual Time Steps



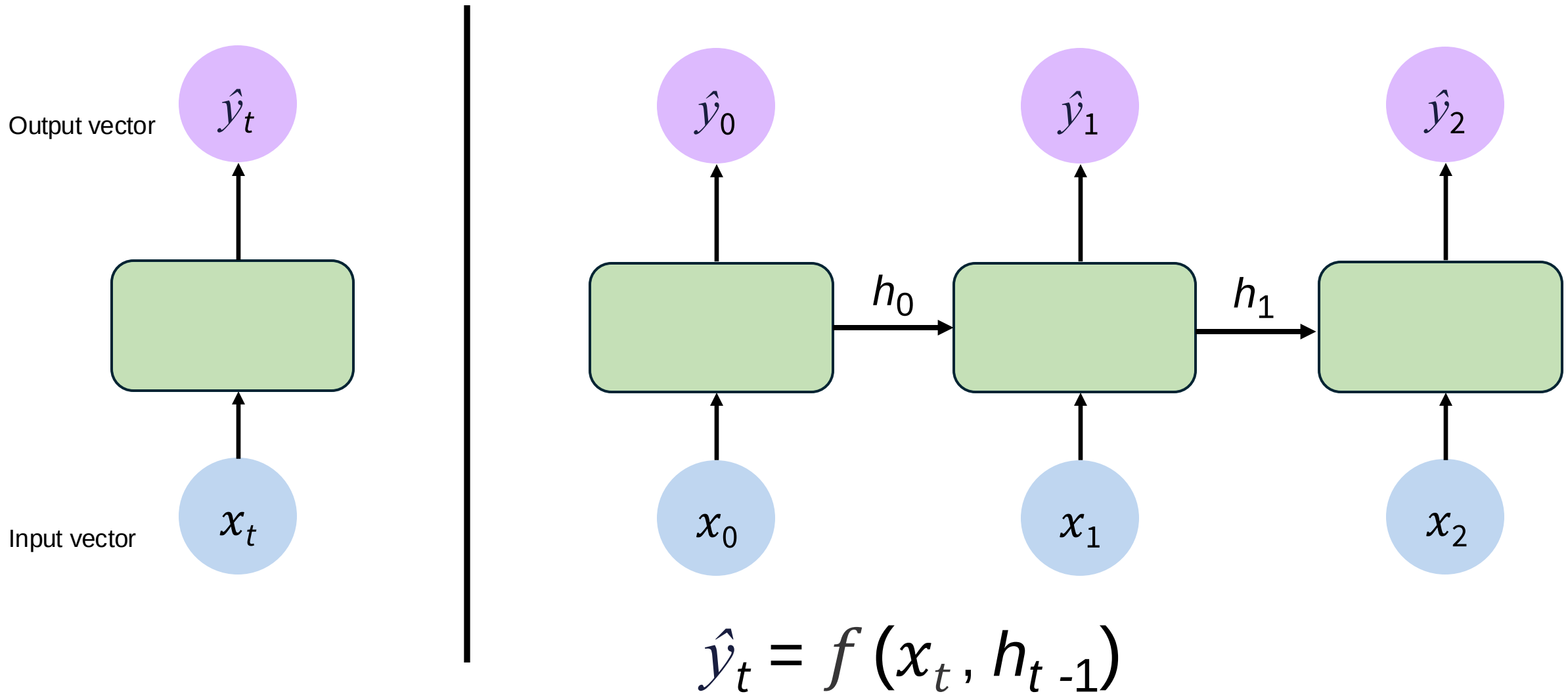
Handling Individual Time Steps



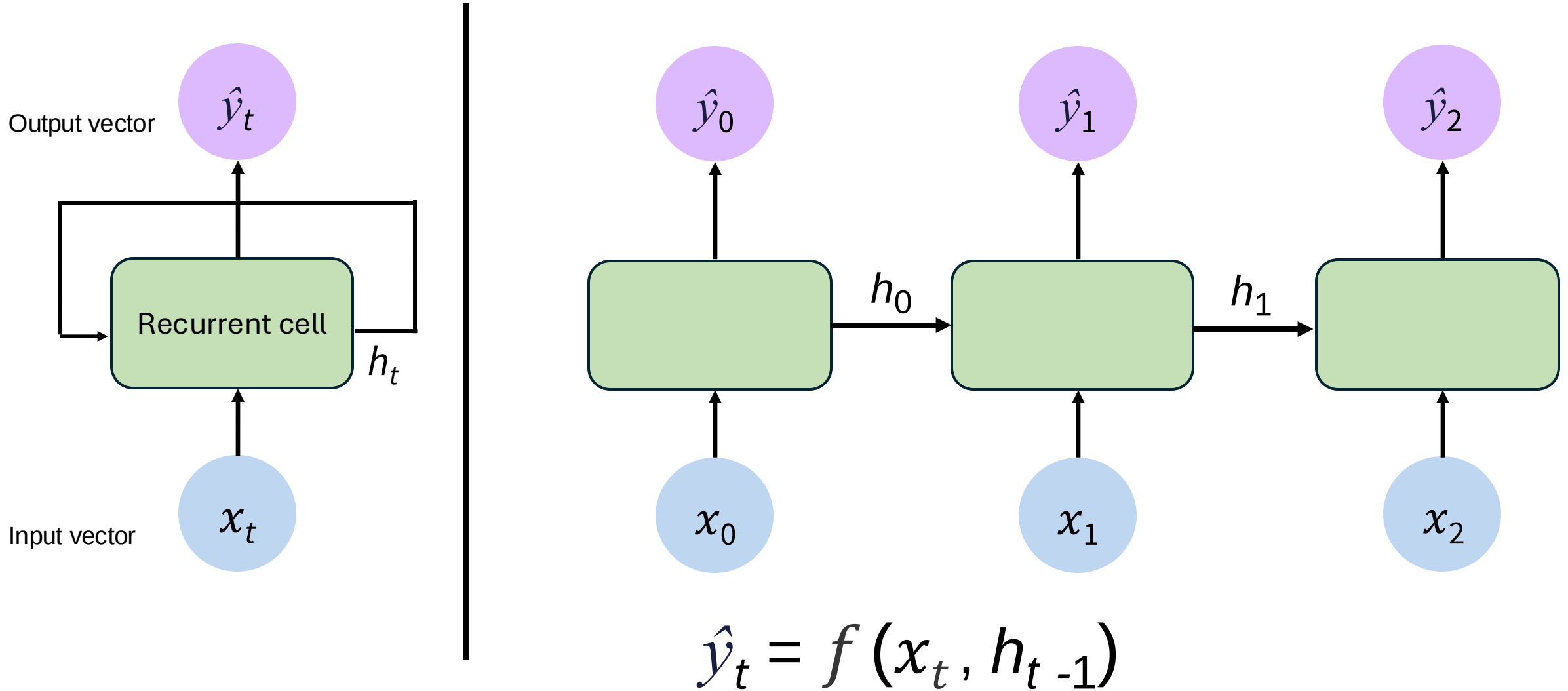
Handling Individual Time Steps - dependence



Neurons with Recurrence (hidden state)



Neurons with Recurrence

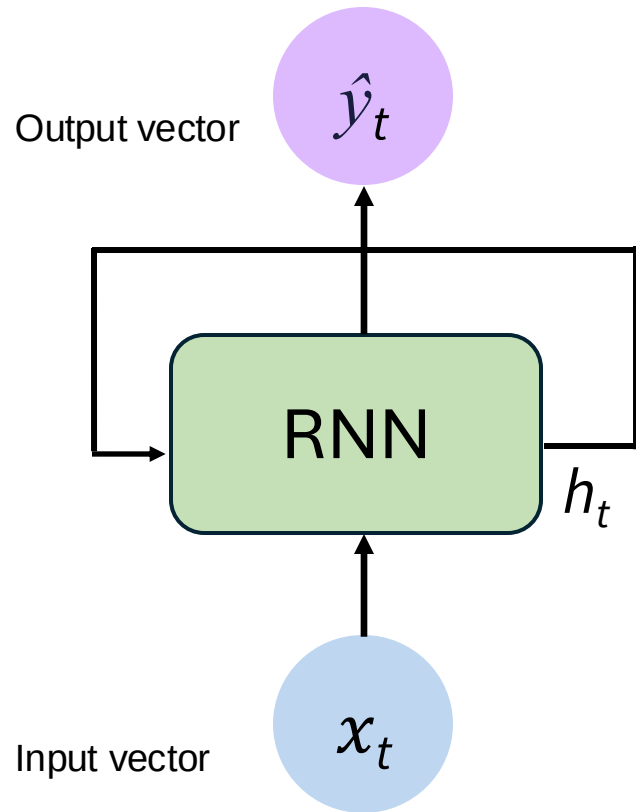


Neurons with Recurrence

- Concrete example of tokens per timestep:



Recurrent Neural Networks (RNNs)



Apply a **recurrence relation** at every time step to process a sequence:

$$\boxed{h_t} = \boxed{f_w}(\boxed{x_t}, \boxed{h_{t-1}})$$

cell state function with weights W input old state

RNNs have a **state**, h_t that is updated **at each time step** as a sequence is processed

RNN Intuition

```
my_rnn = RNN()
```

```
hidden_state = [0, 0, 0, 0]
```

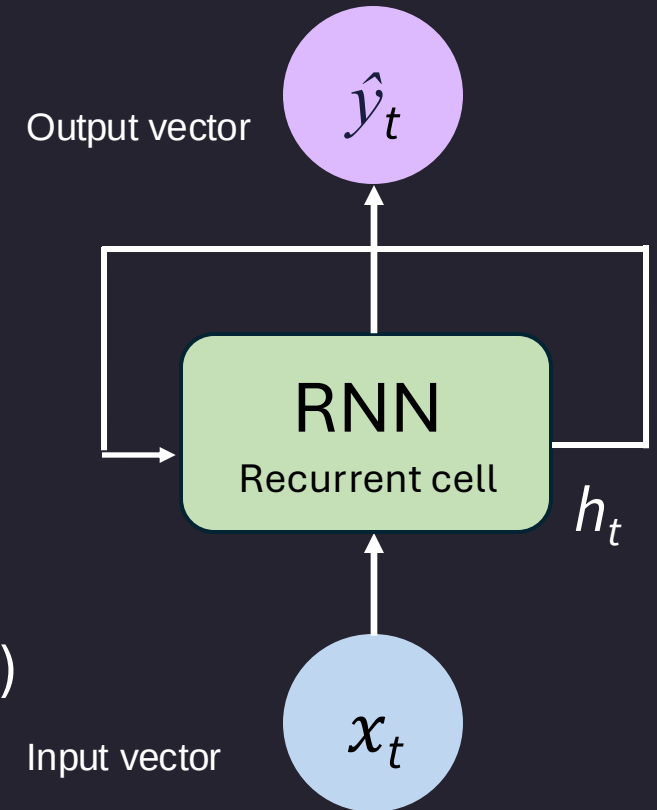
```
sentence = ["I", "love", "recurrent", "neural"]
```

```
for word in sentence:
```

```
    prediction, hidden_state = my_rnn(word, hidden_state)
```

```
next_word_prediction = prediction
```

```
# >>> "networks!"
```



RNN Intuition

```
my_rnn = RNN()
```

```
hidden_state = [0, 0, 0, 0]
```

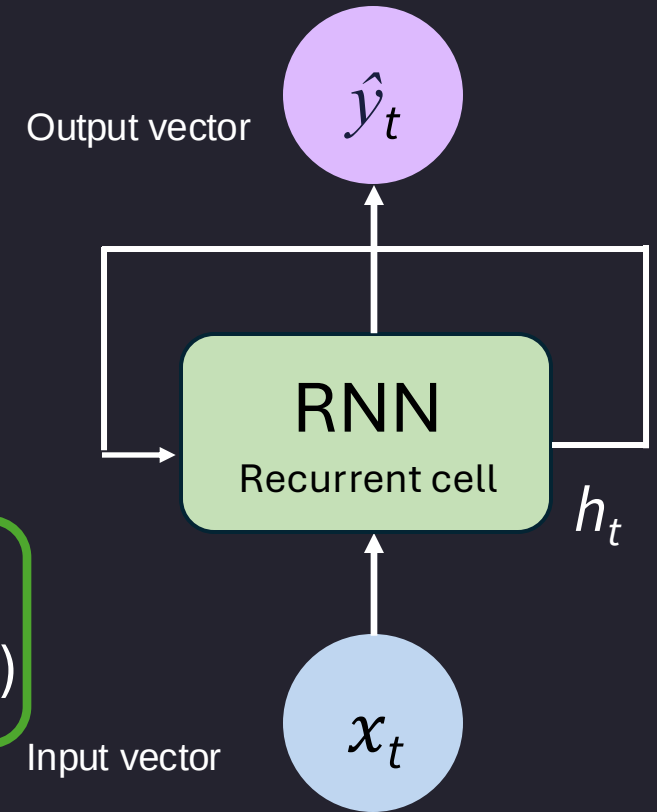
```
sentence = ["I", "love", "recurrent", "neural"]
```

```
for word in sentence:
```

```
    prediction, hidden_state = my_rnn(word, hidden_state)
```

```
next_word_prediction = prediction
```

```
# >>> "networks!"
```



RNN Intuition

```
my_rnn = RNN()
```

```
hidden_state = [0, 0, 0, 0]
```

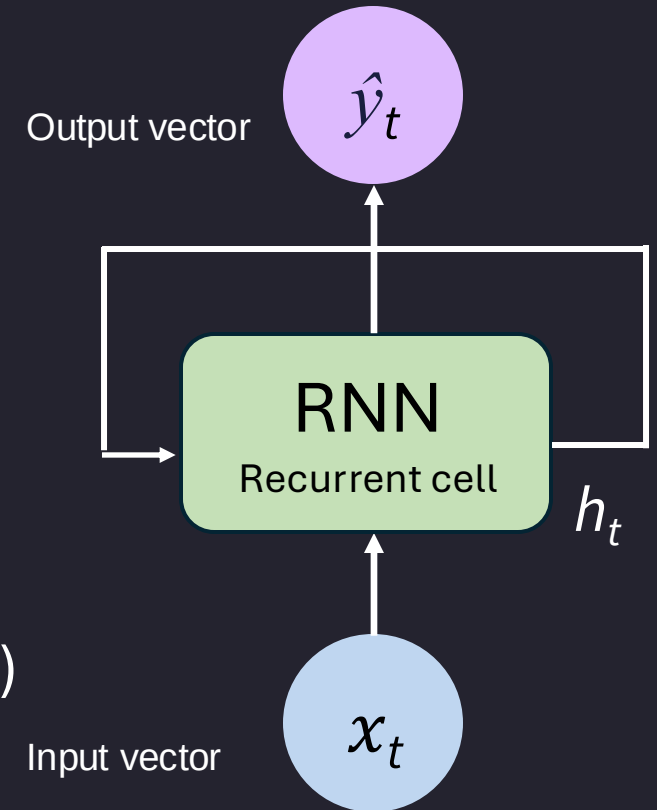
```
sentence = ["I", "love", "recurrent", "neural"]
```

```
for word in sentence:
```

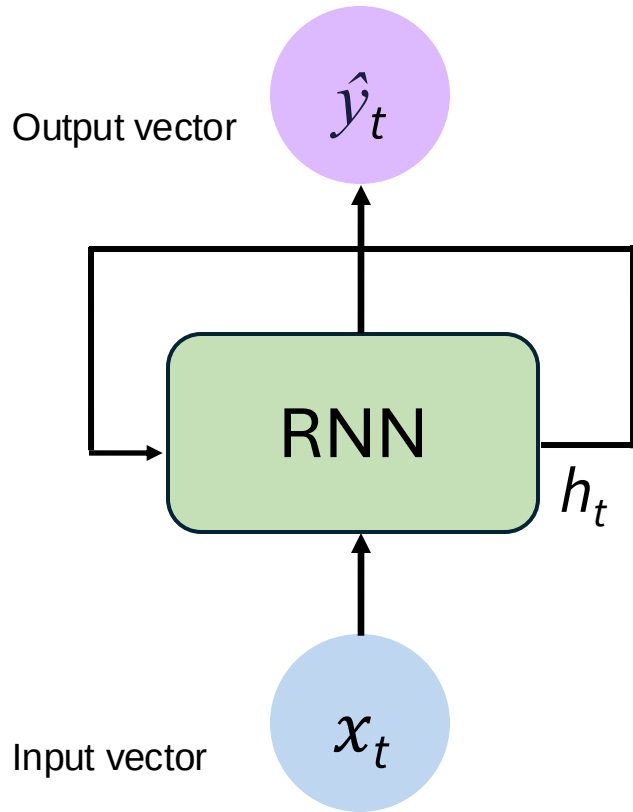
```
    prediction, hidden_state = my_rnn(word, hidden_state)
```

```
next_word_prediction = prediction
```

```
# >>> "networks!"
```



RNN State Update and Output



Output Vector

$$\hat{y}_t = \mathbf{W}_{hy}^T h_t$$

Update Hidden State

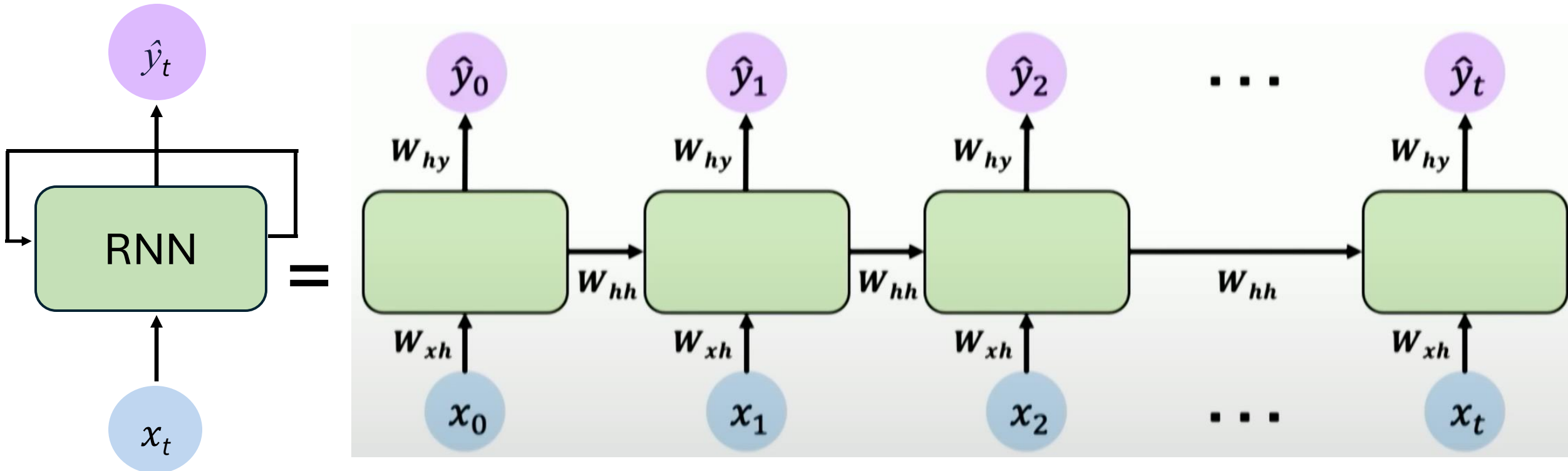
$$h_t = \tanh(\mathbf{W}_{hh}^T h_{t-1} + \mathbf{W}_{xh}^T x_t)$$

Input vector:

$$x_t$$

RNN Computational Graph across Time

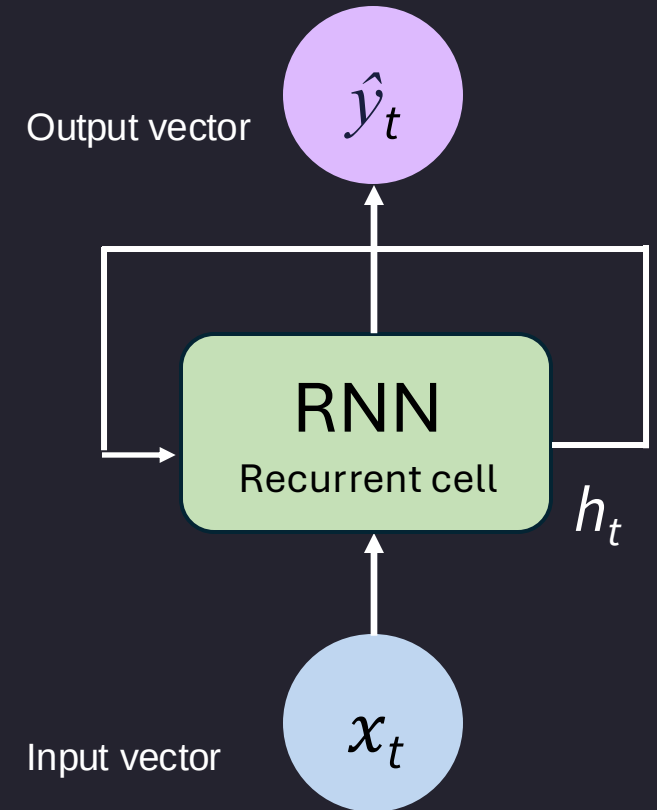
Re-use the **same weight matrices** at each time step



RNNs from Scratch

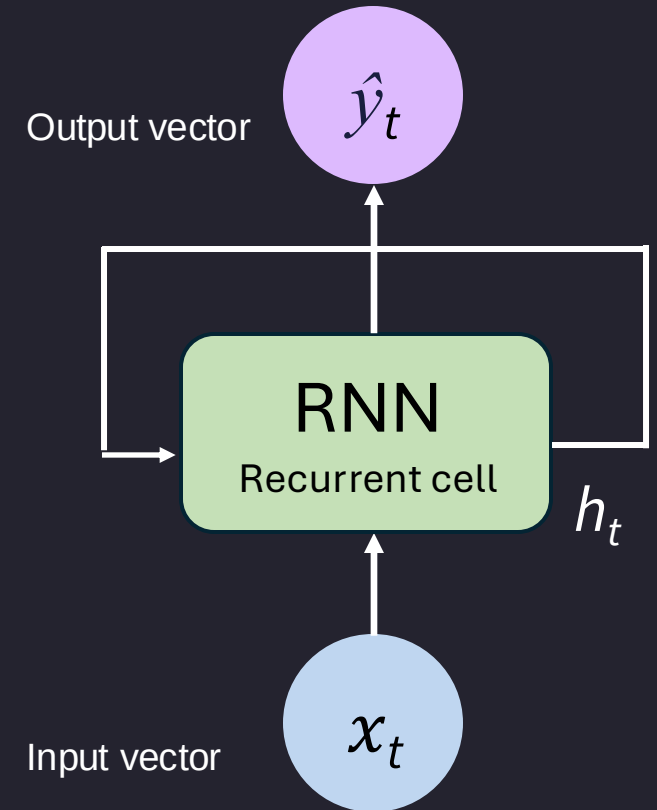
```
class MyRNNCell(tf.keras.layers.Layer):
    def __init__(self, rnn_units, input_dim, output_dim):
        super(MyRNNCell, self).__init__()
        # initialise weight matrices
        self.W_xh = self.add_weight([rnn_units, input_dim])
        self.W_hh = self.add_weight([rnn_units, rnn_units])
        self.W_hy = self.add_weight([output_dim, rnn_units])
        # initialise hidden state to zeros
        self.h = tf.zeros([rnn_units, 1])

    def call(self, x):
        # update the hidden state
        self.h = tf.math.tanh( self.W_hh * self.h + self.W_xh * x )
        # compute the output
        output = self.W_hy * self.h
        # return the current output and hidden state
        return output, self.h
```



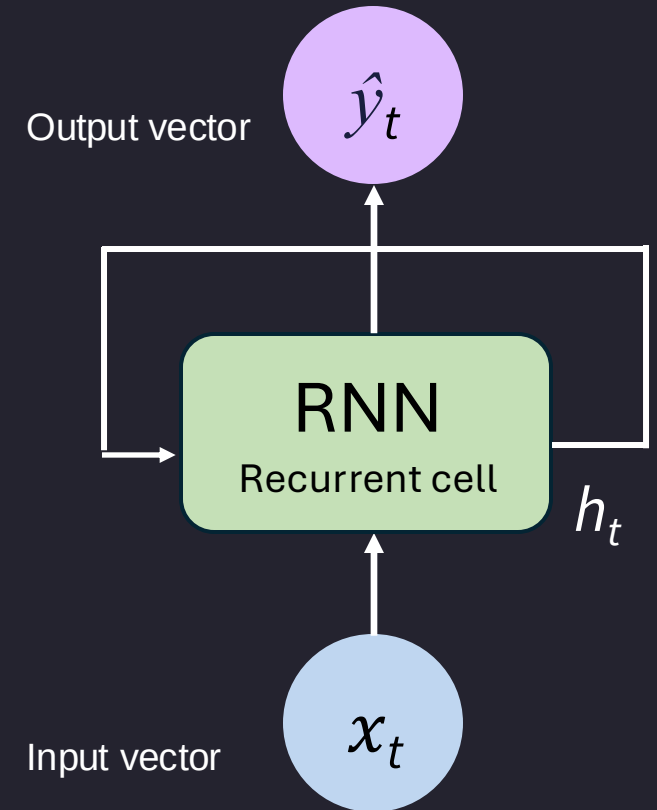
RNNs from Scratch

```
class MyRNNCell(tf.keras.layers.Layer):  
    def __init__(self, rnn_units, input_dim, output_dim):  
        super(MyRNNCell, self).__init__()  
        # initialise weight matrices  
        self.W_xh = self.add_weight([rnn_units, input_dim])  
        self.W_hh = self.add_weight([rnn_units, rnn_units])  
        self.W_hy = self.add_weight([output_dim, rnn_units])  
        # initialise hidden state to zeros  
        self.h = tf.zeros([rnn_units, 1])  
  
    def call(self, x):  
        # update the hidden state  
        self.h = tf.math.tanh( self.W_hh * self.h + self.W_xh * x )  
        # compute the output  
        output = self.W_hy * self.h  
        # return the current output and hidden state  
        return output, self.h
```



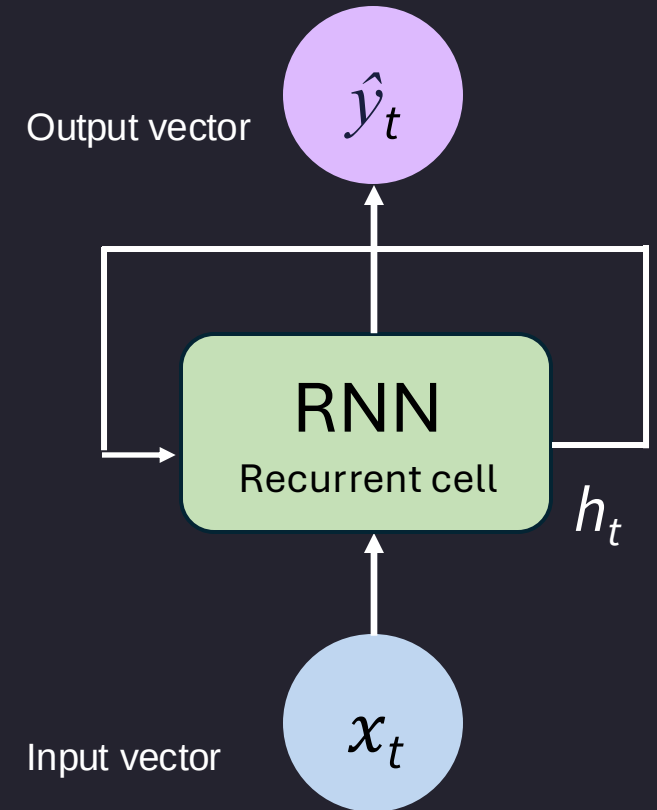
RNNs from Scratch

```
class MyRNNCell(tf.keras.layers.Layer):  
    def __init__(self, rnn_units, input_dim, output_dim):  
        super(MyRNNCell, self).__init__()  
        # initialise weight matrices  
        self.W_xh = self.add_weight([rnn_units, input_dim])  
        self.W_hh = self.add_weight([rnn_units, rnn_units])  
        self.W_hy = self.add_weight([output_dim, rnn_units])  
        # initialise hidden state to zeros  
        self.h = tf.zeros([rnn_units, 1])  
  
    def call(self, x):  
        # update the hidden state  
        self.h = tf.math.tanh( self.W_hh * self.h + self.W_xh * x )  
        # compute the output  
        output = self.W_hy * self.h  
        # return the current output and hidden state  
        return output, self.h
```



RNNs from Scratch

```
class MyRNNCell(tf.keras.layers.Layer):  
    def __init__(self, rnn_units, input_dim, output_dim):  
        super(MyRNNCell, self).__init__()  
        # initialise weight matrices  
        self.W_xh = self.add_weight([rnn_units, input_dim])  
        self.W_hh = self.add_weight([rnn_units, rnn_units])  
        self.W_hy = self.add_weight([output_dim, rnn_units])  
        # initialise hidden state to zeros  
        self.h = tf.zeros([rnn_units, 1])  
  
    def call(self, x):  
        # update the hidden state  
        self.h = tf.math.tanh( self.W_hh * self.h + self.W_xh * x )  
        # compute the output  
        output = self.W_hy * self.h  
        # return the current output and hidden state  
        return output, self.h
```



Problems: long term dependencies

- “**France** is where I grew up, but I now live in Boston. I speak fluent ____.”
- In order to predict ‘French’ as the next word, you would need to remember the very first word of this sequence (France being the clue), not the nearest word (Boston).
- Long term dependencies could be subject to vanishing gradient problems (multiplying small numbers together each time step), or limited memory).

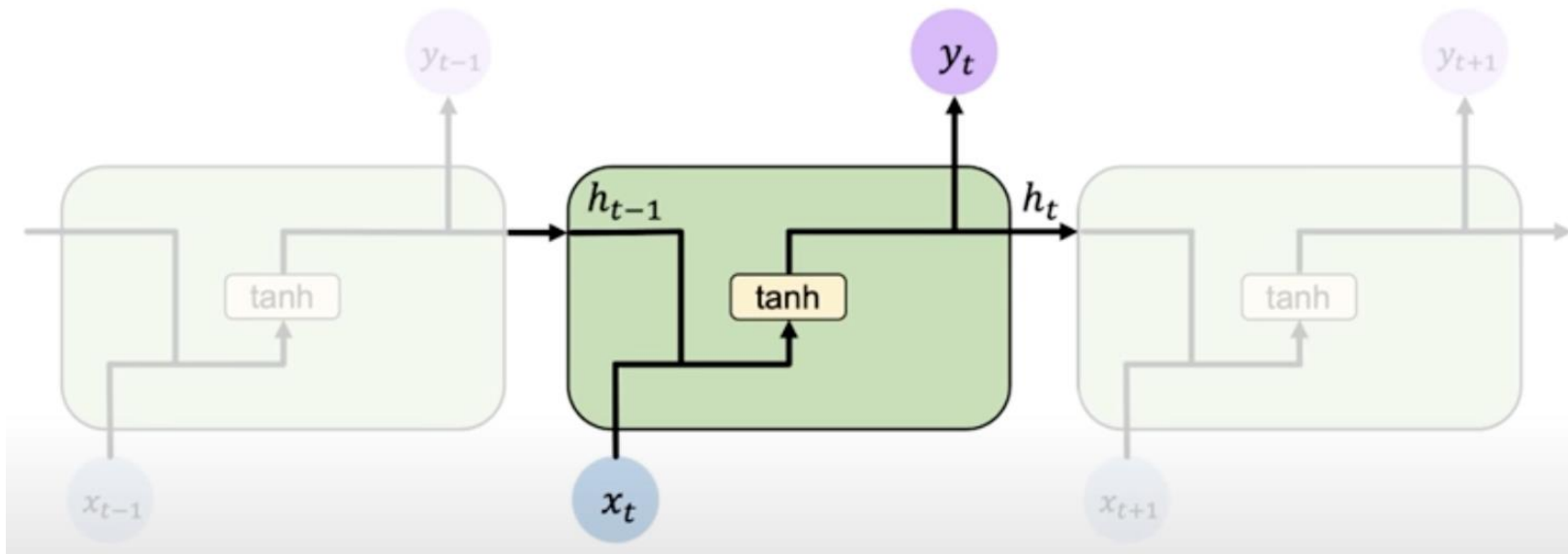
Long-Short Term Memory (LSTMs)

Enter LSTMs

- Long-Short Term Memory (LSTMs) control information via **gates**:
 - Forget Gate: Decides what information to discard from the previous timestep.
 - Input Gate: Decides what new information to store.
 - Output Gate: Decides what to pass to the next timestep.
- Better than RNNs at capturing long-term dependencies due to their gating mechanisms.
- They are still sequential in nature – compressing context (memory) each timestep.
- **They can struggle with very long-range dependencies due to vanishing gradients.**

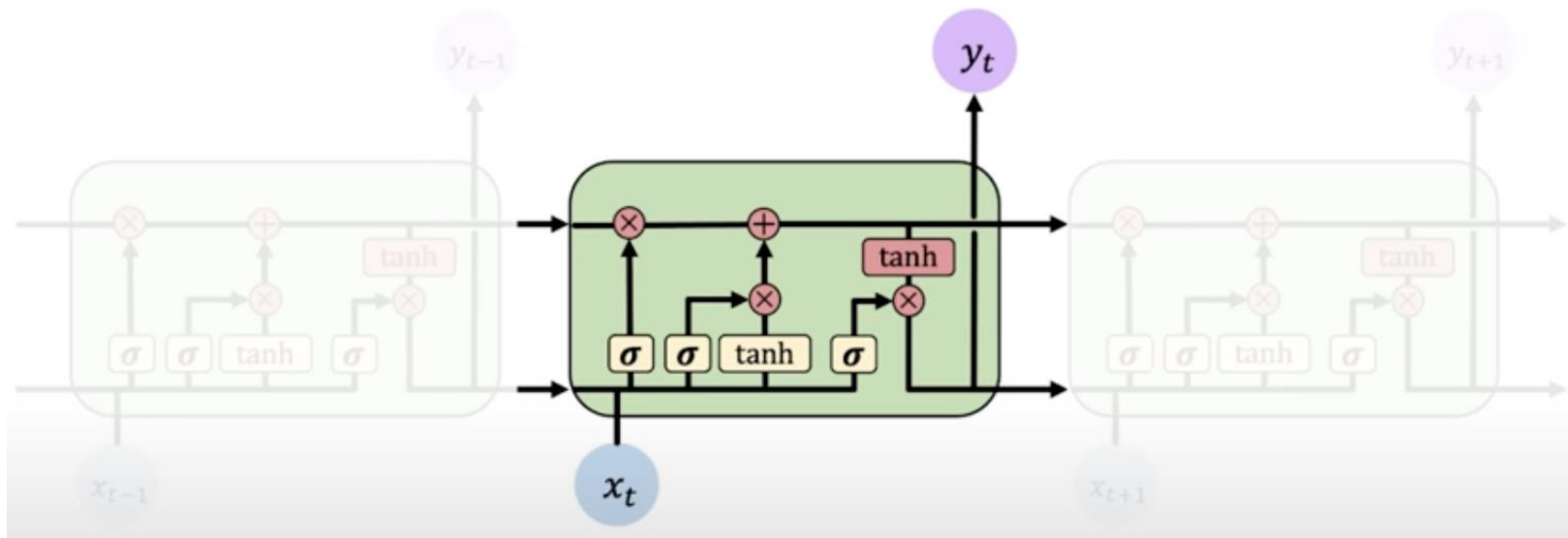
Standard RNN

In a standard RNN, repeating modules contain a **simple computation node**



Long-Short Term Memory (LSTMs)

LSTM modules contain **computational blocks** that **control information flow**

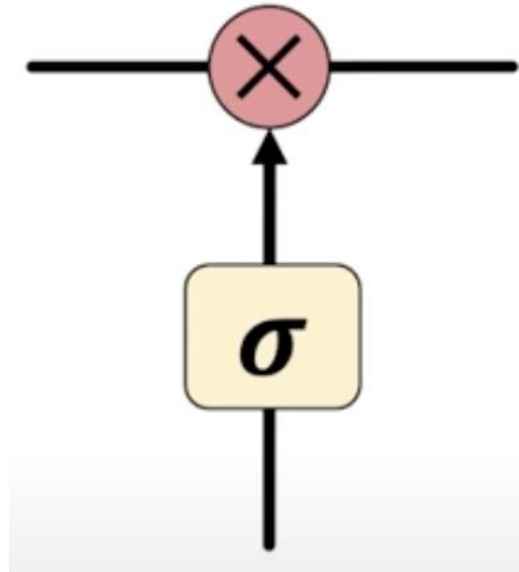


LSTM cells are able to track information through many timesteps

```
tf.keras.layers.LSTM(num_units)
```

Long-Short Term Memory (LSTMs)

Information is **added** or **removed** through structures called **gates**



Gates optionally let information through, for example via a sigmoid neural net layer and pointwise multiplication

Long-Short Term Memory (LSTMs)

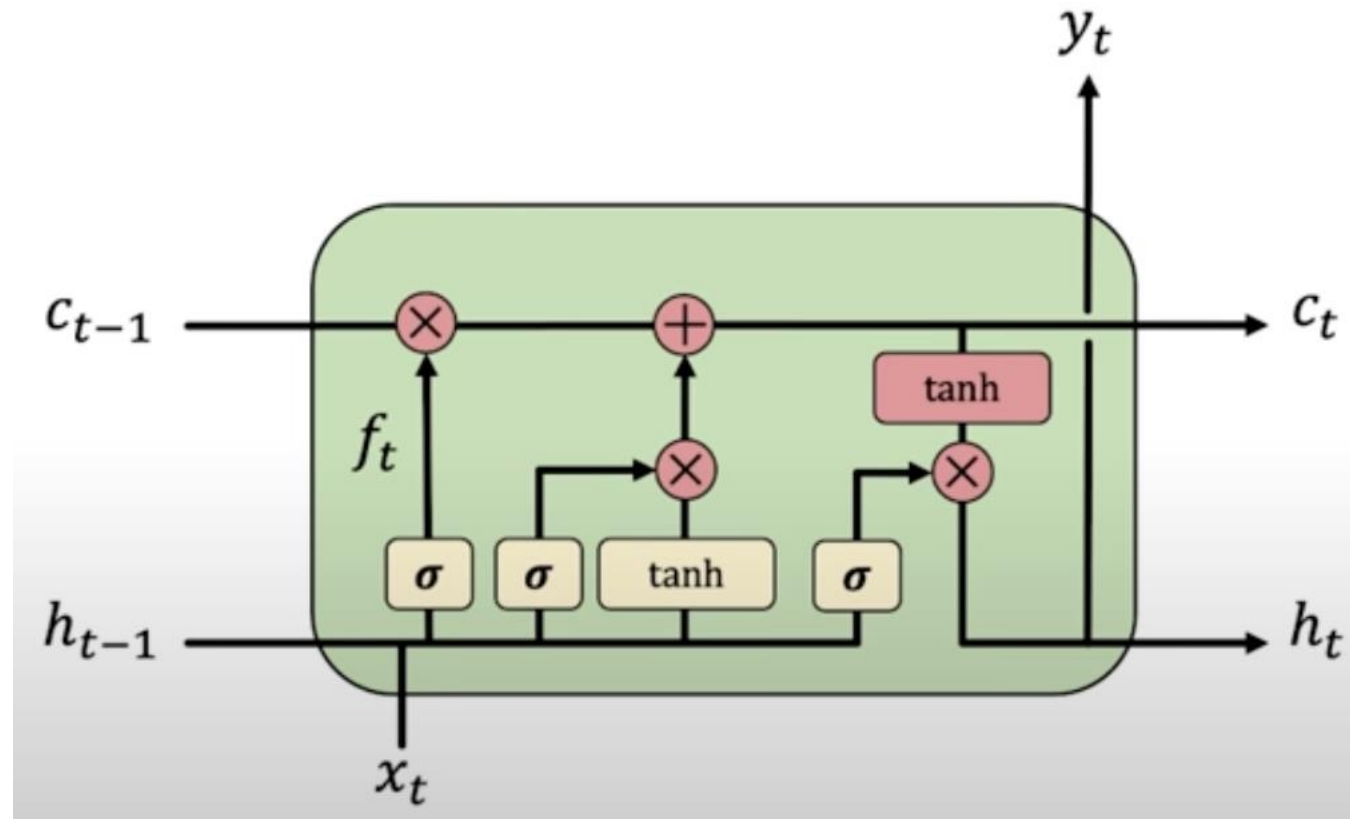
How do LSTMs work?

1) Forget

2) Store

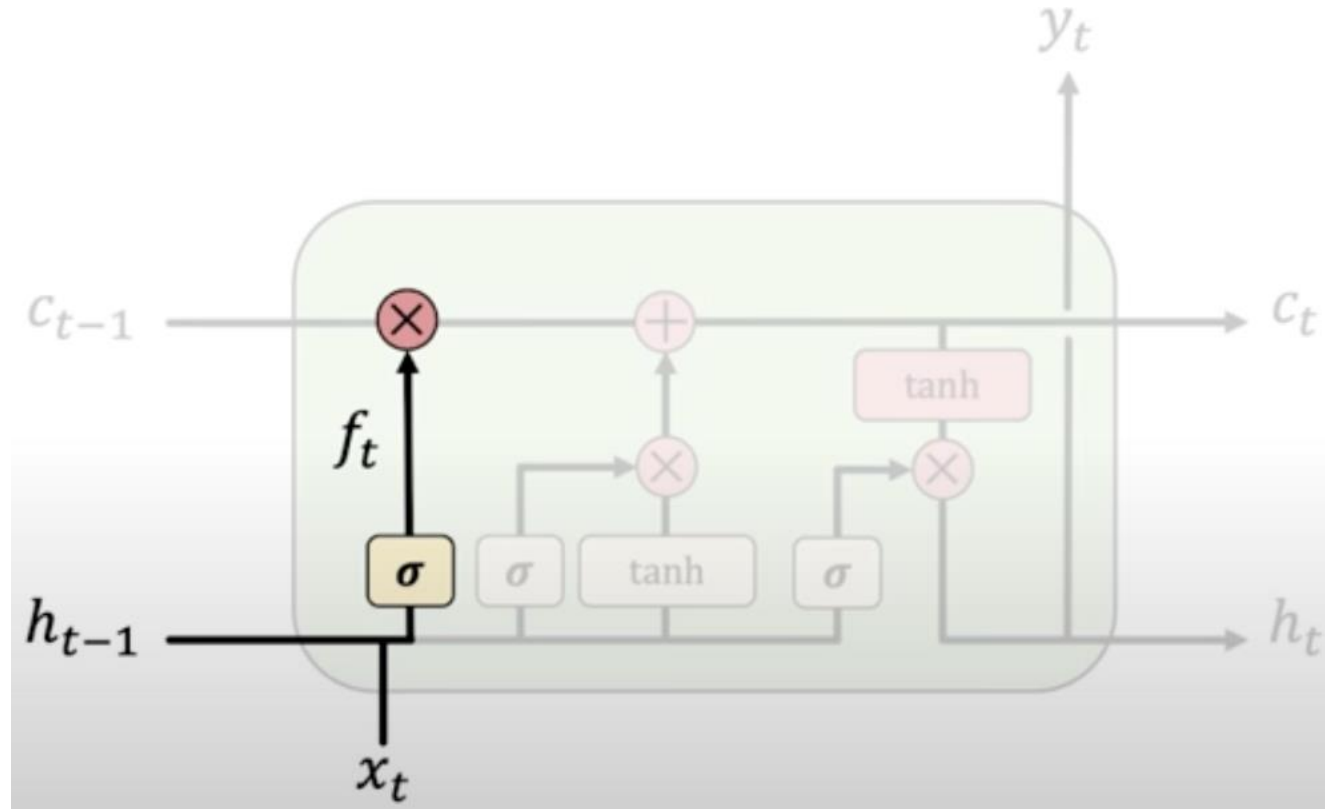
3) Update

4) Output



Long-Short Term Memory (LSTMs)

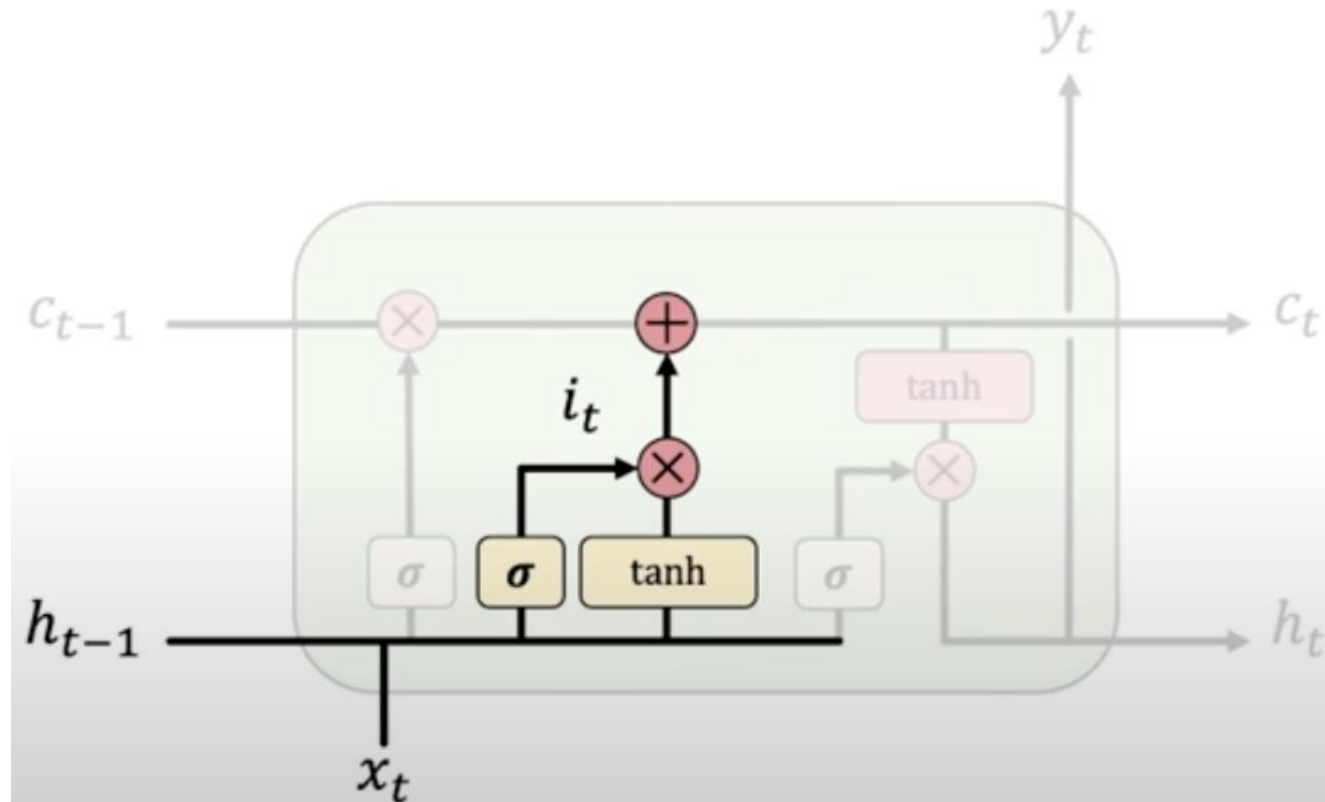
1) **Forget** 2) Store 3) Update 4) Output
LSTMs **forget irrelevant** parts of the previous state



Long-Short Term Memory (LSTMs)

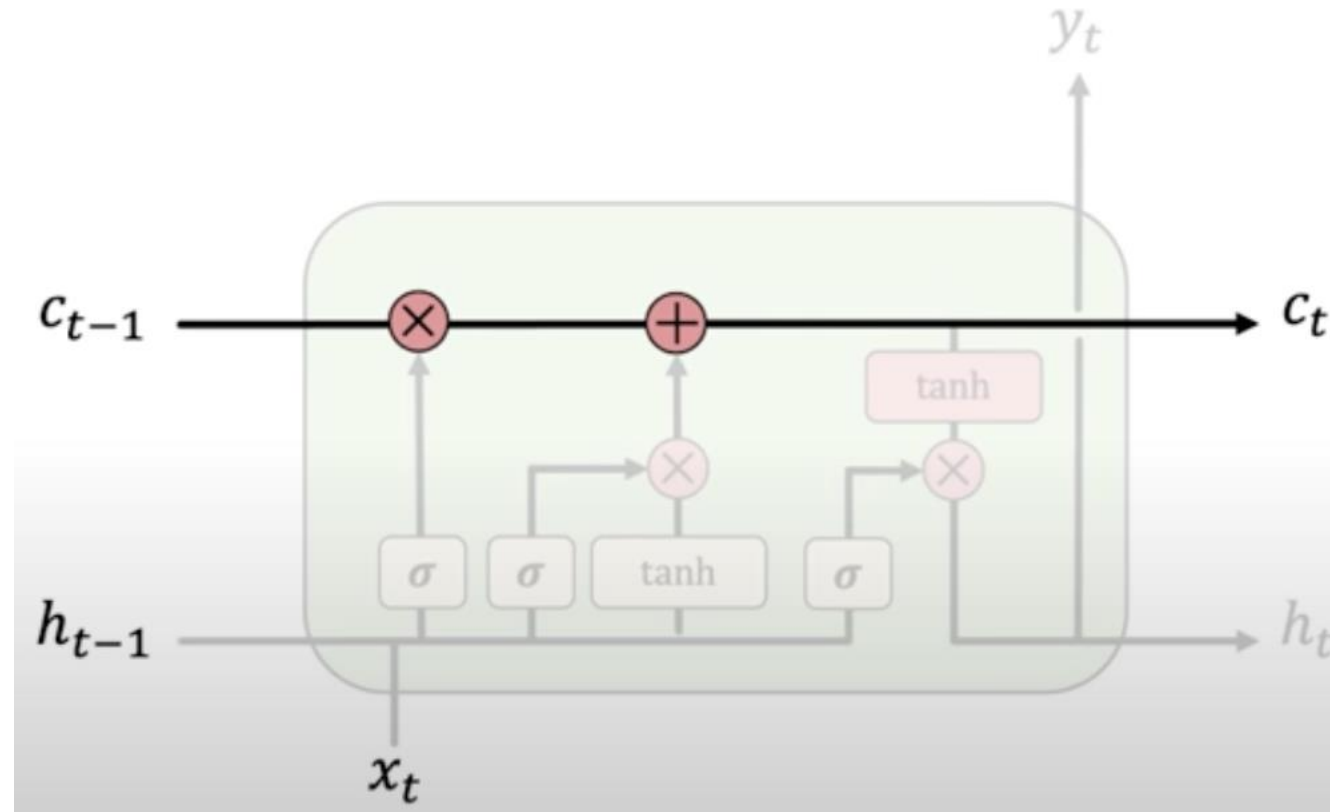
1) Forget **2) Store** 3) Update 4) Output

LSTMs **store relevant** new information into the cell state



Long-Short Term Memory (LSTMs)

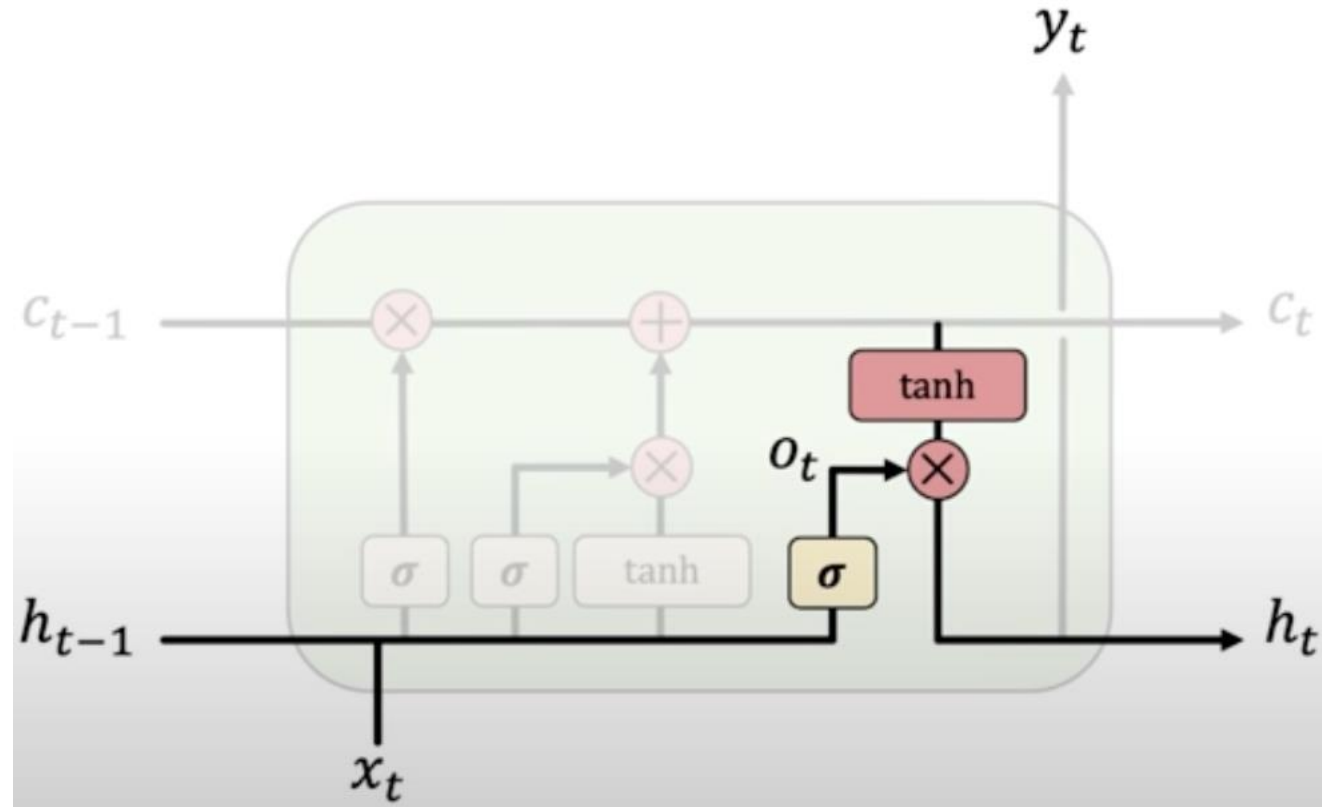
- 1) Forget 2) Store **3) Update** 4) Output
LSTMs **selectively update** cell state values



Long-Short Term Memory (LSTMs)

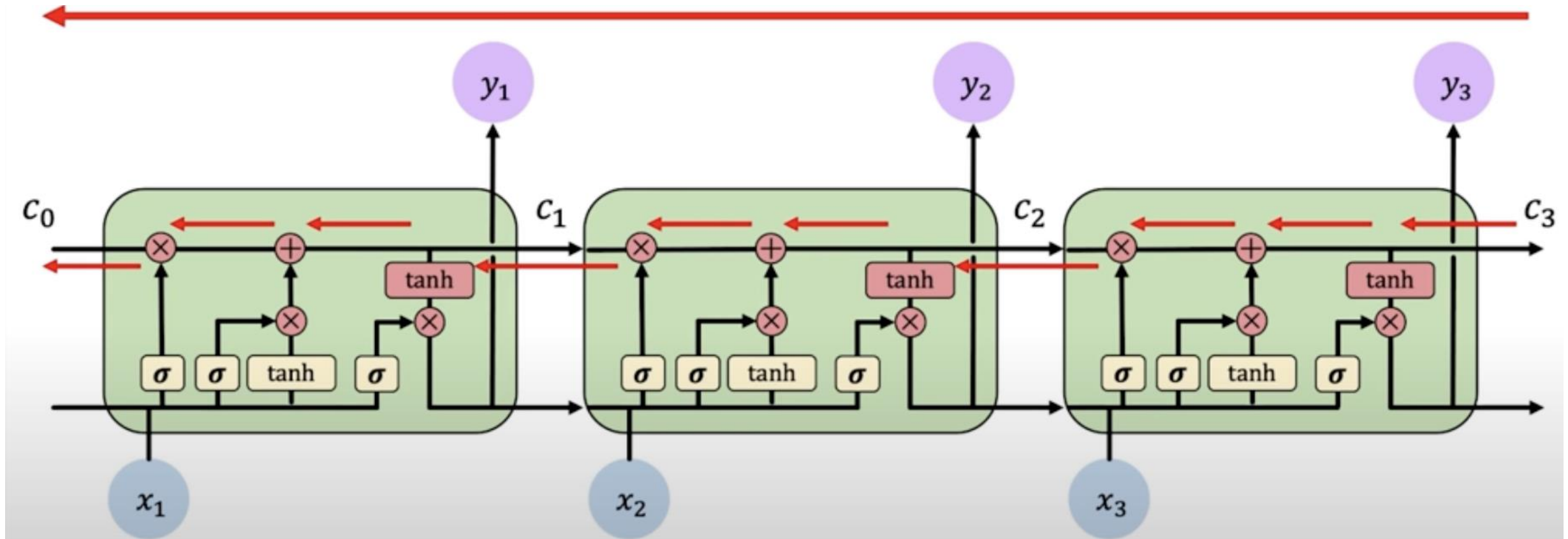
1) Forget 2) Store 3) Update 4) **Output**

The **output gate** controls what information is sent to the next time step



LSTM Gradient Flow

Uninterrupted gradient flow!



LSTMs: Key Concepts

- Maintain a **separate cell state** from what is output
- Use **gates** to control the **flow of information**
 - **Forget** gate gets rid of irrelevant information
 - **Store** relevant information from current input
 - **Selectively update** cell state
 - **Output** gate returns a filtered version of the cell state
- Backpropagation through time with **uninterrupted gradient flow**

RNNs to LSTMs



- LSTMs improve upon traditional RNNs due to their gating mechanism.
- However, still sequential in nature – earlier information may be forgotten as not deemed important at a given timestep.
- **Compression of Information:** As the sequence grows longer, the hidden state may struggle to retain information from the distant past due to vanishing gradients or limited memory capacity. Due to sequential dependency, nearby points in the reduced space often correspond to nearby timesteps in the sequence.
- For a sentence like “The quick brown fox jumps,” the hidden state vectors may form a trajectory where: The hidden state at “The” starts neutral. Subsequent hidden states shift incrementally as new words modify the context.
- **What if there was a way to eliminate the need for recurrence entirely?**

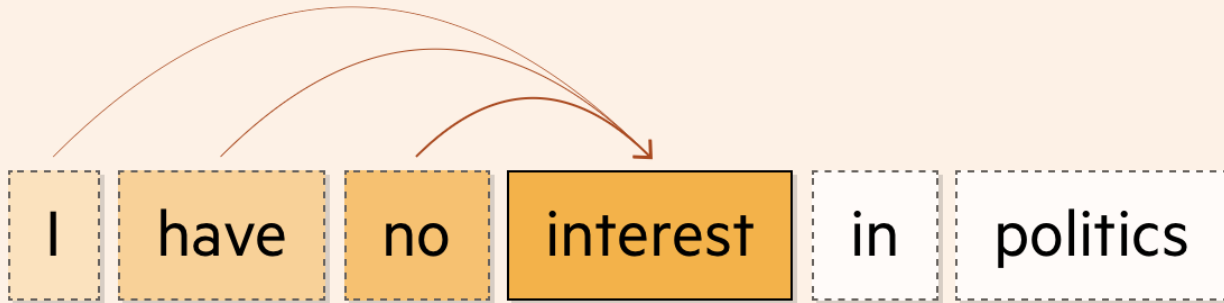
Attention

Enter Attention

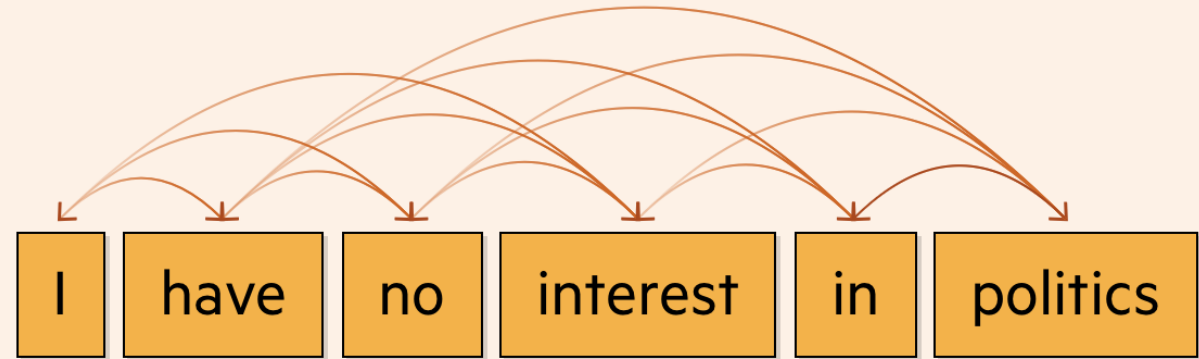
- The attention mechanism enables the model to weigh the importance of different input tokens for generating each output token.
- Directly models relationships between tokens, even over long distances.
- Attention scores are calculated for each input token relative to the current output token, normalised with softmax, and use these scores to compute weighted combinations of input embeddings.
- Can enhance RNNs (e.g., LSTMs with attention) for tasks like machine translation.
- However, **just attention alone** does not have the ability to solve the scalability or parallelism issues of RNNs.

Enter Attention

RNNs/LSTMs scan each word in a sentence **sequentially**, updating a hidden state which compresses context (memory).



With self-attention, the **transformer computes weights for all the words in a sentence at the same time (global context)**.



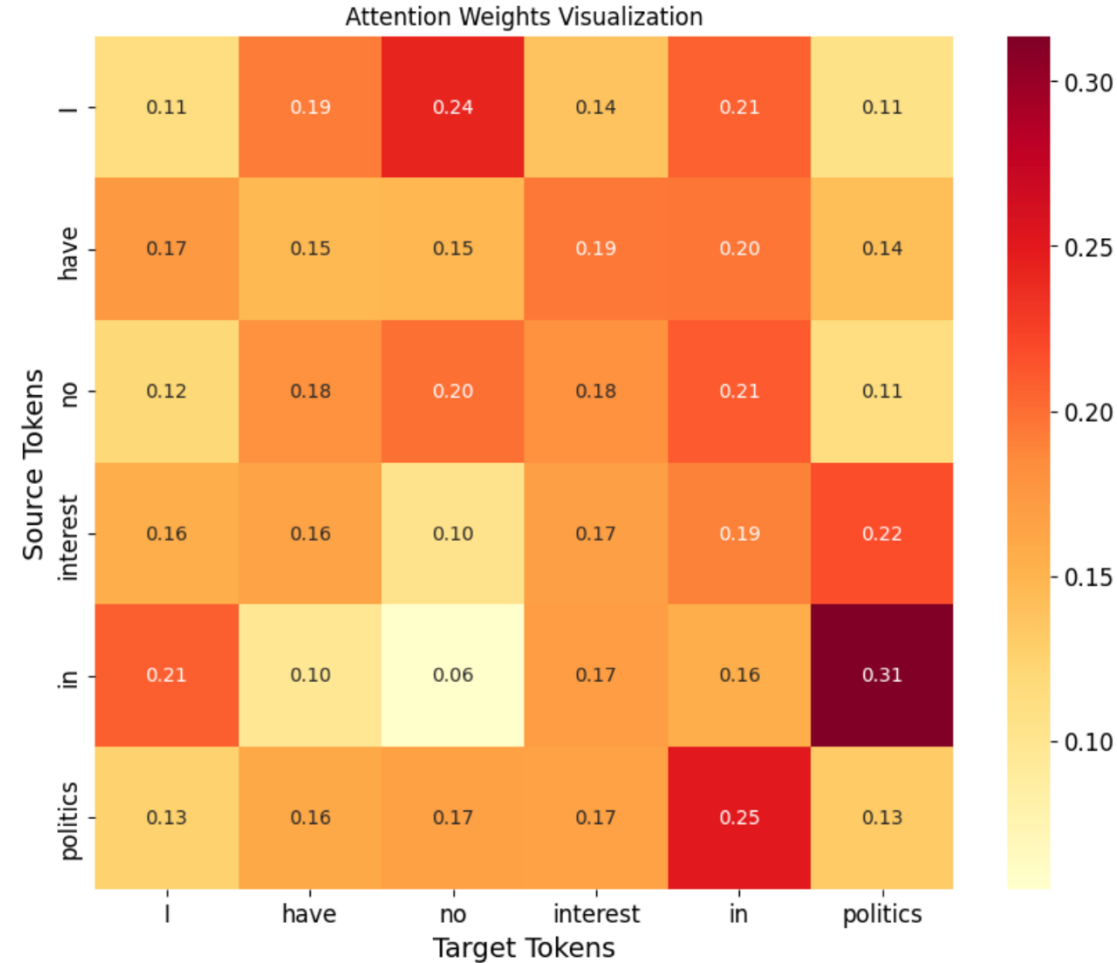
Example of attention weights (single-head)



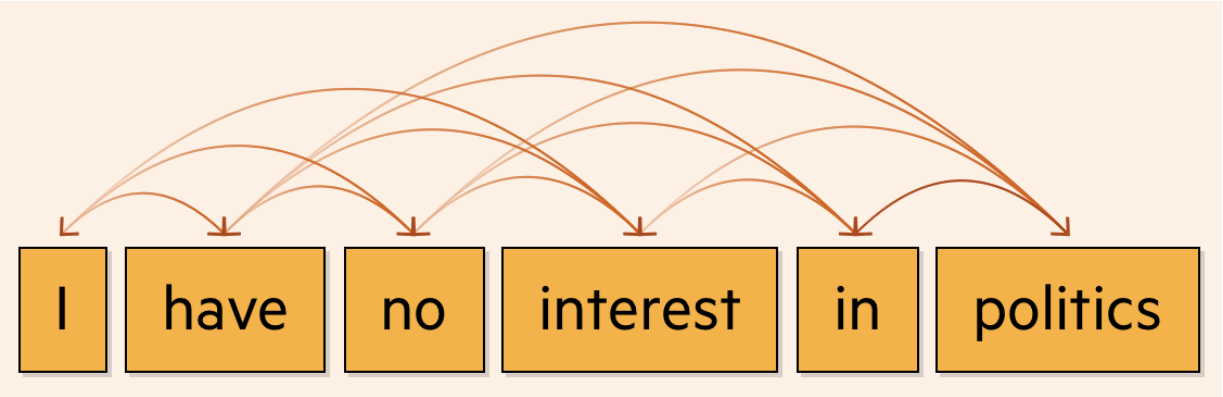
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Attention is calculated from a query and key

- **A query** represents the “question” or “information request” a token has in the context of the entire sequence. It’s used to search for relevant information in other tokens (through their keys).
- **A key** represents the information available in a token that can potentially be attended to by the queries of other tokens. It’s essentially the “attribute” or “feature” that a token offers for attention.



Attention (Q, K and V)



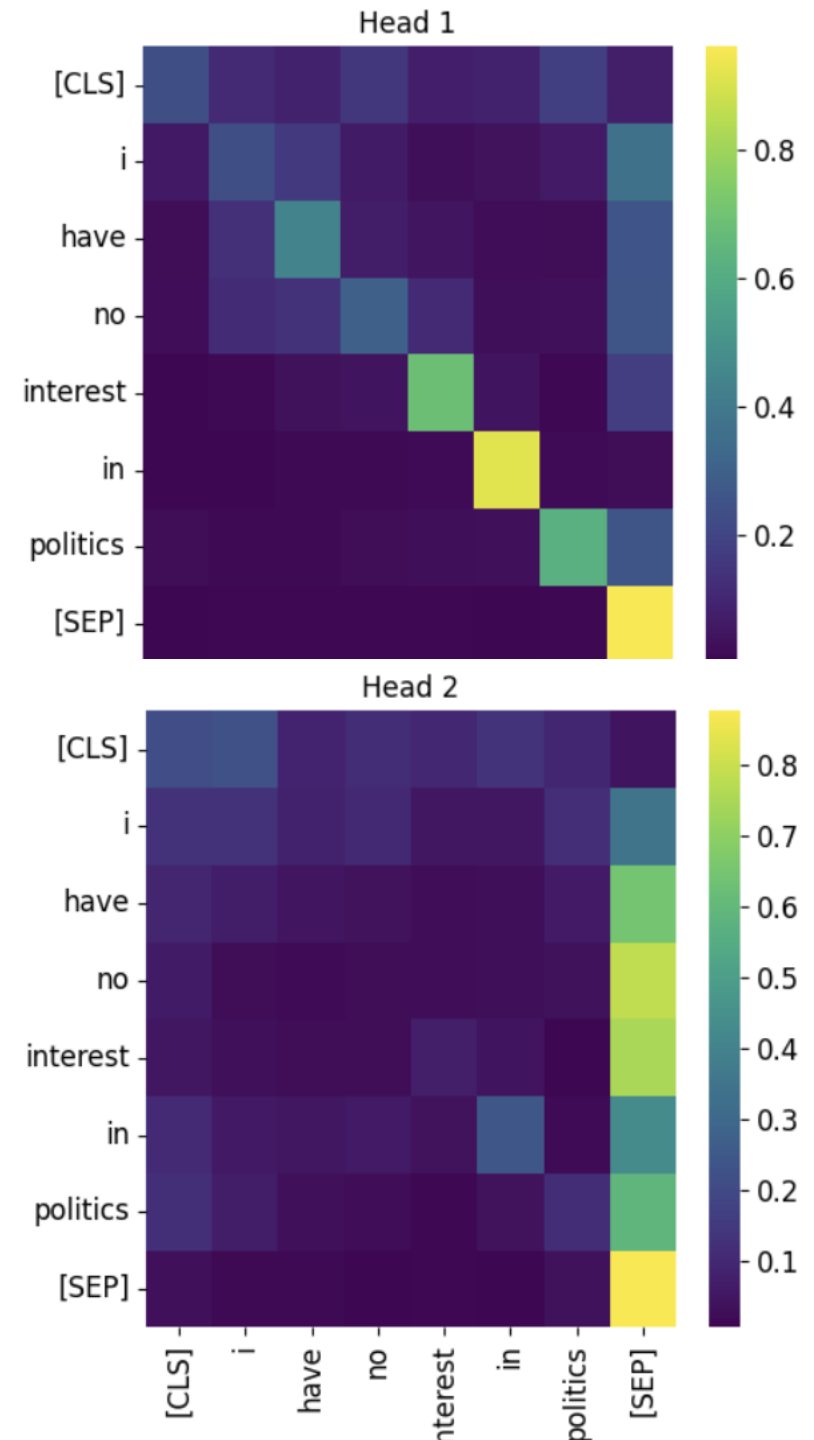
- **Query (Q):** The **vector** representing what you're looking for.
- Queries are asked for each token:
- The **query for "interest"** might be looking for related words like "in" and "politics"
- The **query for "no"** might be looking for the word it's modifying ("interest")
- The **query for "in"** might be looking for both "interest" before it and "politics" after it

```
1 # Example query vectors for each word position:
2
3 I:      [0.3, -0.1, 0.2, 0.4]
4 have:   [-0.2, 0.5, -0.3, 0.1]
5 no:     [0.4, -0.2, 0.1, -0.3]
6 interest: [0.1, 0.3, 0.5, -0.1]
7 in:     [-0.3, 0.2, -0.1, 0.4]
8 politics: [0.2, 0.4, -0.2, 0.3]
```

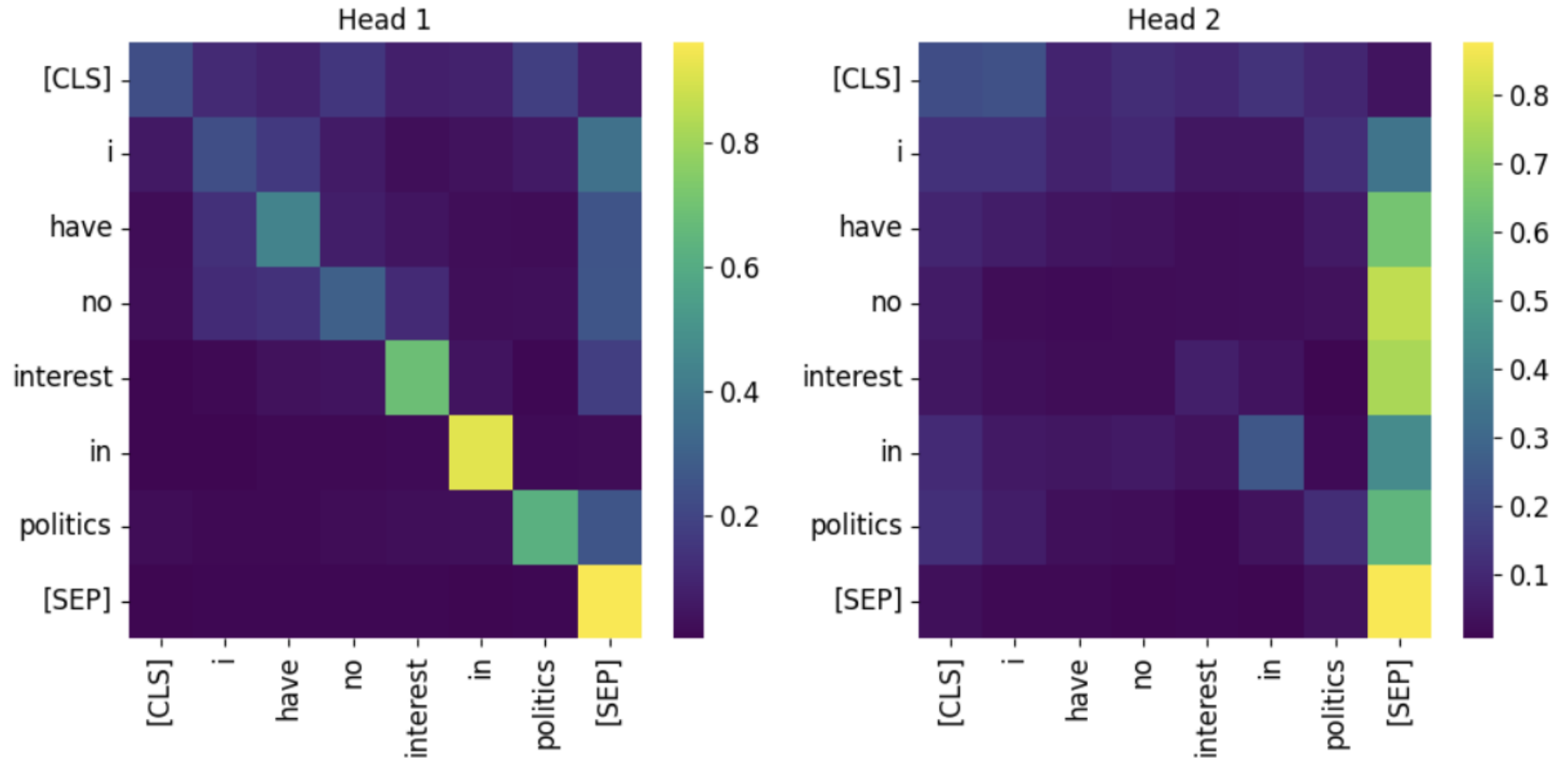
Multi-head Attention

- Each ‘head’ is a weight matrix showing how much ‘attention’ each token pays to others.
- **Head 1** might focus on **semantic** relationships (e.g., “politics” attends to “interest”).
- **Head 2** might focus on **syntactic** relationships (e.g., “no” attends to “interest”).

[CLS] and [SEP] are special tokens used by BERT to mark the start and end of the sequence.



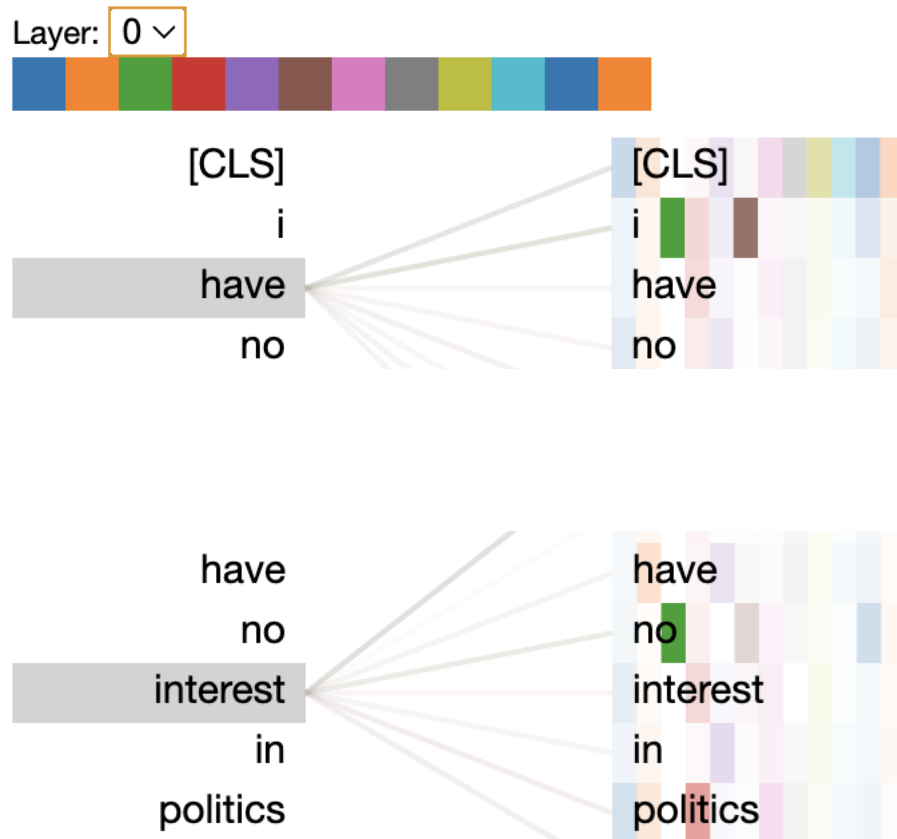
Multi-head Attention



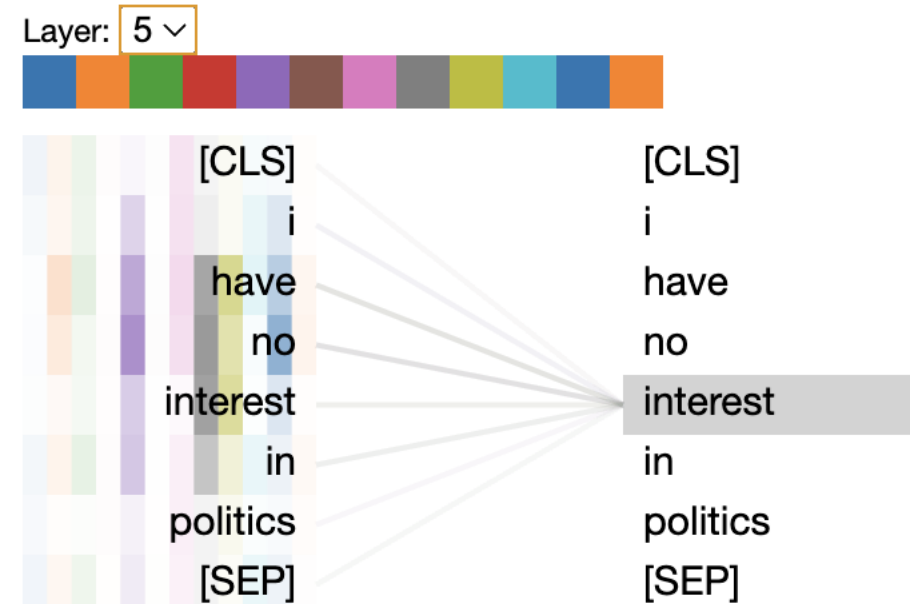
[CLS] and [SEP] are special tokens used by BERT to mark the start and end of the sequence.

Visualising the Attention Scores with BertViz

Lower layers = Syntax & grammar

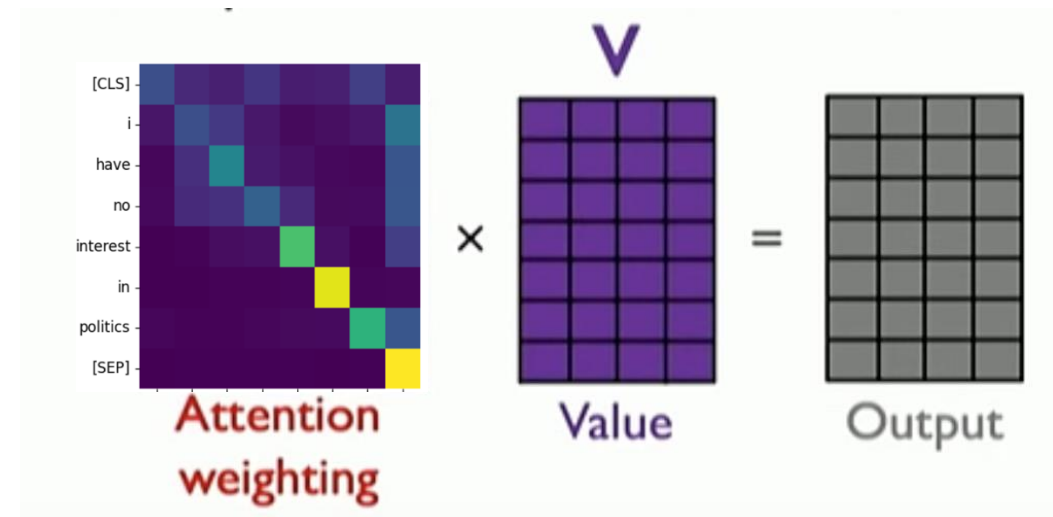
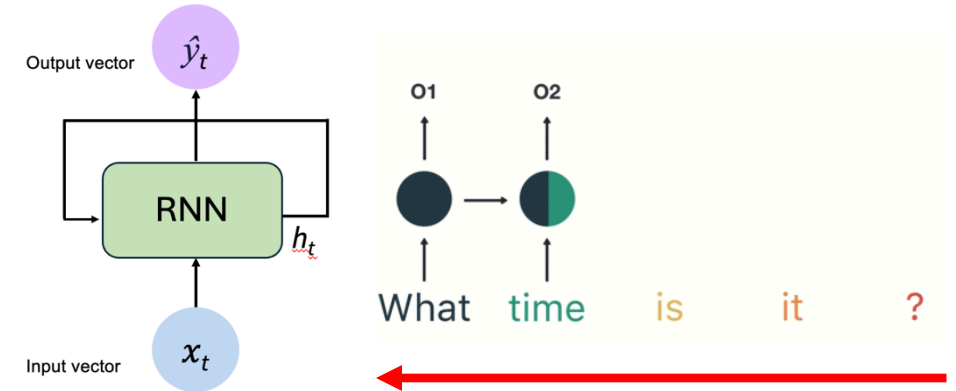


Higher layers = Semantic meaning



Recurrent architecture vs Transformers

- RNNs/LSTMs need to perform a **linear transformation** at **Each Time Step**:
- At each time step, an RNN or LSTM applies a linear transformation (typically in the form of a matrix multiplication) to the current token's input and the previous hidden state – **Backpropagation Through Time (BPTT)**.
- In RNNs/LSTMs, BPTT is used to update the hidden state, which is then used for the next time step.
- In a **Transformer**, a single set of **linear transformations** (like those for queries W_Q , keys W_K , and values W_V) is **applied to all tokens at once** for the entire sequence (not sequentially step by step like RNNs and LSTMs).
- The self-attention mechanism calculates the relationship (attention weightings) between all pairs of tokens in a single pass.



Transformers

Transformation steps we will see

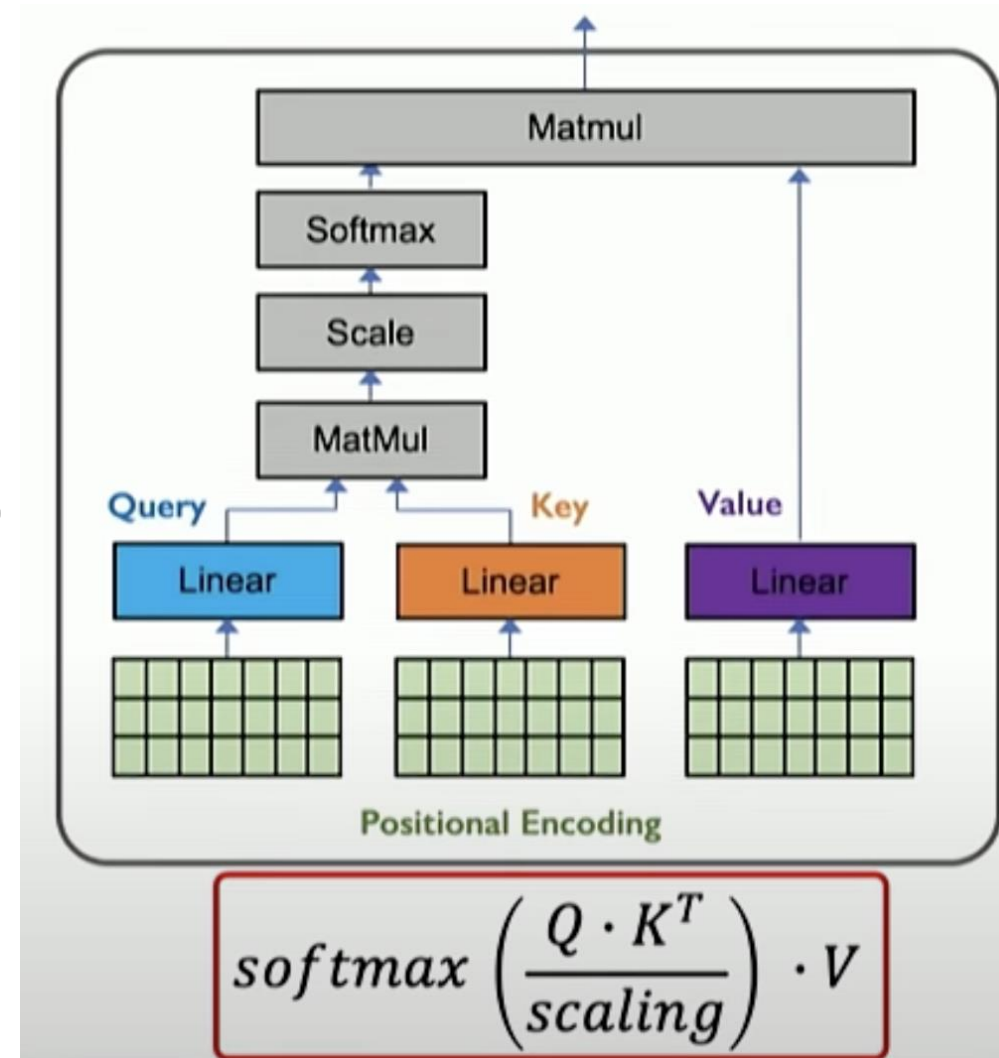
- Token embedding & positional encoding
 - Transformation is the raw text -> dense vectors
- Self-Attention (Q, K, V, Softmax, Weighted Sum)
 - Transformation is the embeddings -> context—aware representations
- Multi-head attention
 - Transformation is the expansion of attention principle across multiple perspectives

Transformer steps

1. Positional Encoding
2. Extract **query**, **key**, **value** for search
3. Compute attention weighting (**Q** and **K**)
4. Extract features with high attention



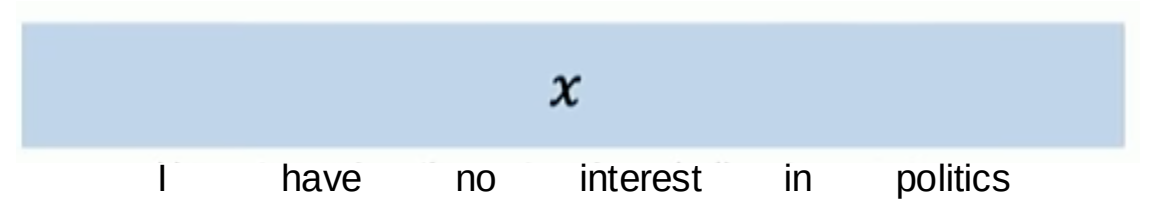
Feed Forward Network



The entire **Self-Attention** mechanism is a **learnable transformation** that takes raw embeddings and **modifies them dynamically** based on context.

Transformer steps

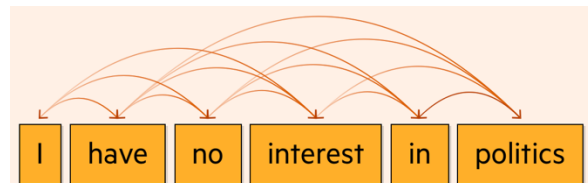
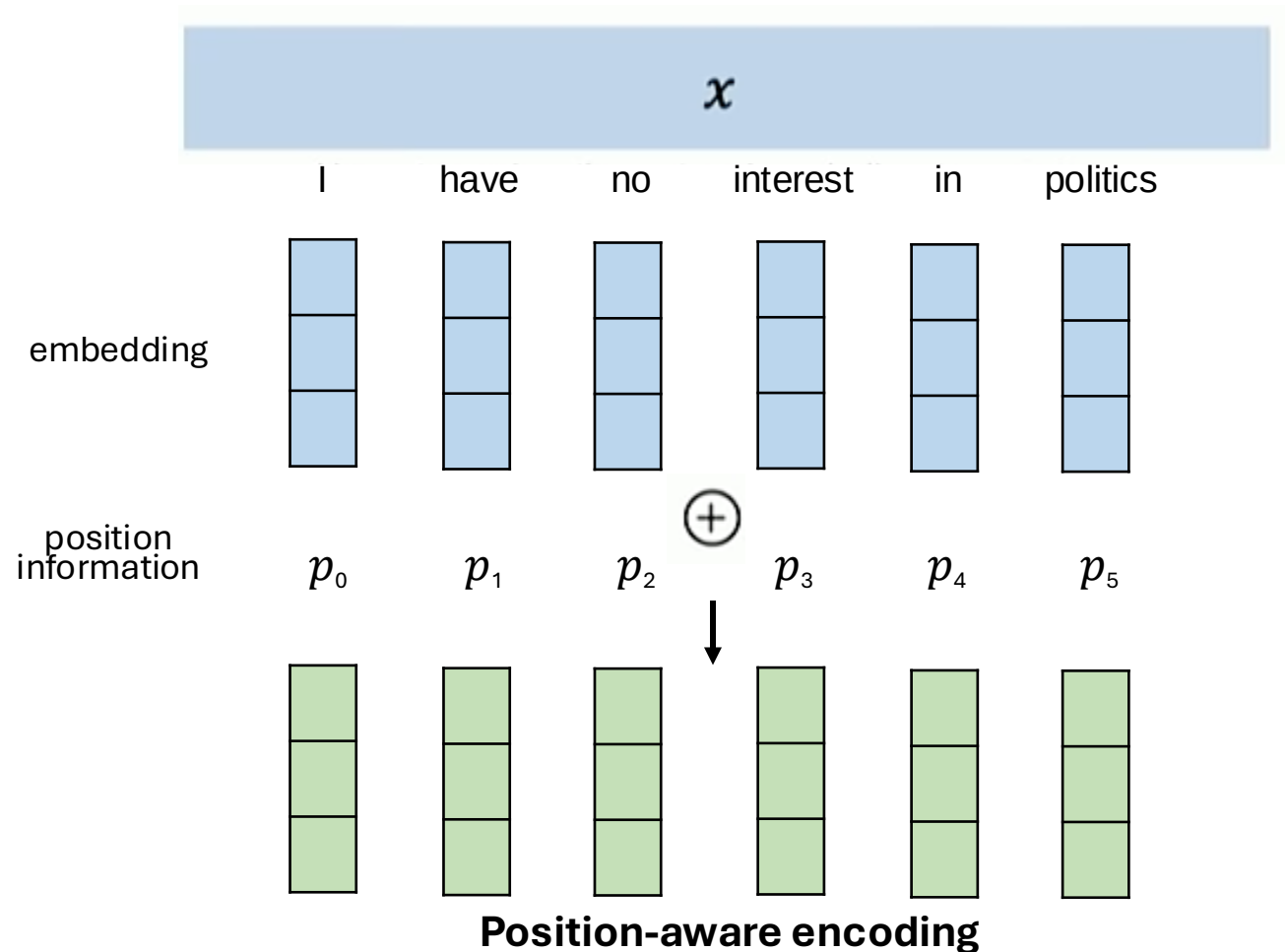
Goal: identify and attend to most important features in input



1. Encode position information

Goal: identify and attend to most important features in input

1. Encode **position** information



Data is fed in all at once! (no recurrence)
Need to encode position information to understand order

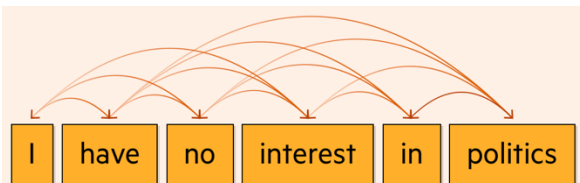
1. Encode position information

Goal: identify and attend to most important features in input

1. Encode **position** information

```
1 # Embedding matrix for tokens
2
3 embeddings = torch.tensor([
4     [0.0000, 1.0000, 0.0000, 1.0000], # "I"
5     [0.8415, 0.5403, 0.0100, 0.9999], # "have"
6     [0.9093, -0.4161, 0.0200, 0.9998], # "no"
7     [0.1411, -0.9900, 0.0300, 0.9996], # "interest"
8     [-0.7568, -0.6536, 0.0400, 0.9992], # "in"
9     [-0.9589, 0.2837, 0.0500, 0.9988] # "politics"
10 ], dtype=torch.float32)
```

Position-aware encoding

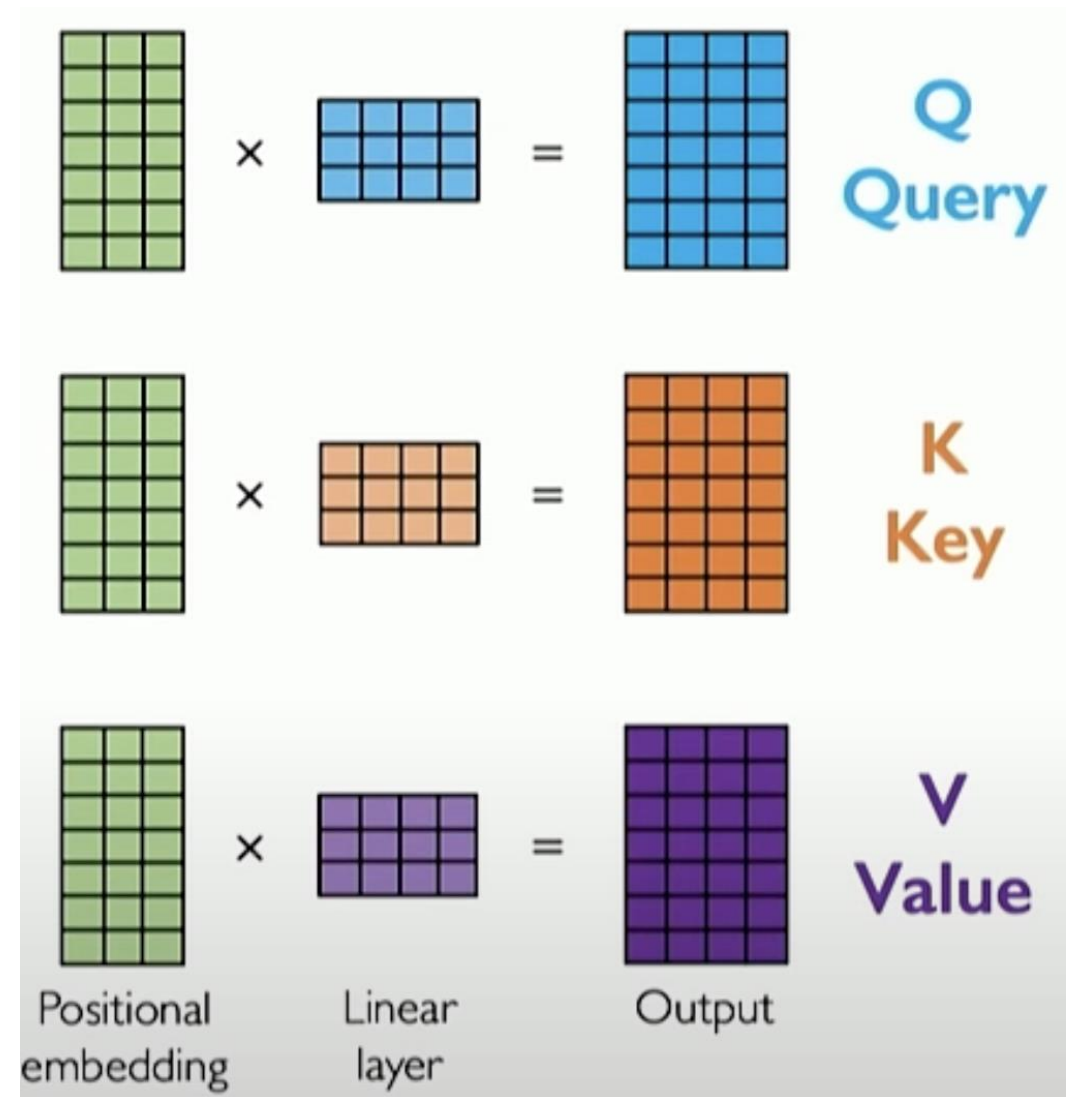


Data is fed in all at once! (no recurrence)
Need to encode position information to understand order

2. Extract Query, Key and Value (Transformation)

Goal: identify and attend to most important features in input

1. Encode **position** information
2. Extract **Q**, **K** and **V** for search



2. Extract Query, Key and Value

```
1 # Embedding matrix for tokens
2
3 embeddings = torch.tensor([
4     [0.0000, 1.0000, 0.0000, 1.0000], # "I"
5     [0.8415, 0.5403, 0.0100, 0.9999], # "have"
6     [0.9093, -0.4161, 0.0200, 0.9998], # "no"
7     [0.1411, -0.9900, 0.0300, 0.9996], # "interest"
8     [-0.7568, -0.6536, 0.0400, 0.9992], # "in"
9     [-0.9589, 0.2837, 0.0500, 0.9988] # "politics"
10 ], dtype=torch.float32)
```

Positional embedding

```
1 W_Q
✓ 0.0s

tensor([[ -1.2279,  0.0316, -0.5274,  1.5270],
        [ -0.3770, -1.3565, -1.5063,  0.0312],
        [  0.3060,  0.3394,  0.1335, -0.9418],
        [ -0.2147,  0.4700,  0.3439, -0.5219]])
```

W_Q

(learned during training)

```
1 Q = embeddings @ W_Q
2 Q
✓ 0.0s

tensor([[ -0.5917, -0.8865, -1.1625, -0.4906],
        [ -1.4486, -0.2330, -0.9125,  0.7706],
        [ -1.1682,  1.0699,  0.4937,  0.8349],
        [ -0.0054,  1.8274,  1.7646, -0.3654],
        [  0.9734,  1.3459,  1.7326, -1.7352],
        [  0.8714,  0.0713,  0.4285, -2.0237]])
```

Q

Query Matrix

```
1 # Embedding matrix for tokens
2
3 embeddings = torch.tensor([
4     [0.0000, 1.0000, 0.0000, 1.0000], # "I"
5     [0.8415, 0.5403, 0.0100, 0.9999], # "have"
6     [0.9093, -0.4161, 0.0200, 0.9998], # "no"
7     [0.1411, -0.9900, 0.0300, 0.9996], # "interest"
8     [-0.7568, -0.6536, 0.0400, 0.9992], # "in"
9     [-0.9589, 0.2837, 0.0500, 0.9988] # "politics"
10 ], dtype=torch.float32)
```

Positional embedding

```
1 W_K
✓ 0.0s

tensor([[ -0.6074,  0.4019, -0.3172, -1.3953],
        [ -0.0930, -1.9480,  1.6627, -0.7314],
        [ -0.2962, -1.4790, -0.7530,  0.2072],
        [ -0.9298, -0.3955, -0.0604, -0.8712]])
```

W_K

(learned during training)

```
1 K = embeddings @ W_K
2 K
✓ 0.0s

tensor([[ -1.0228, -2.3435,  1.6023, -1.6026],
        [ -1.4940, -1.1245,  0.5635, -2.4383],
        [ -1.4491,  0.7511, -1.0557, -1.8312],
        [ -0.9320,  1.5456, -1.7738, -0.3374],
        [ -0.4205,  0.5147, -0.9372,  0.6718],
        [ -0.3875, -1.4070,  0.6778,  0.2707]])
```

K

Key Matrix

3. Compute attention weighting from **Q** and **K**

3. Compute **attention weighting**

3.1 Pairwise similarity **Q** and **K**

3.2 Scaled similarity

3.3 Softmax

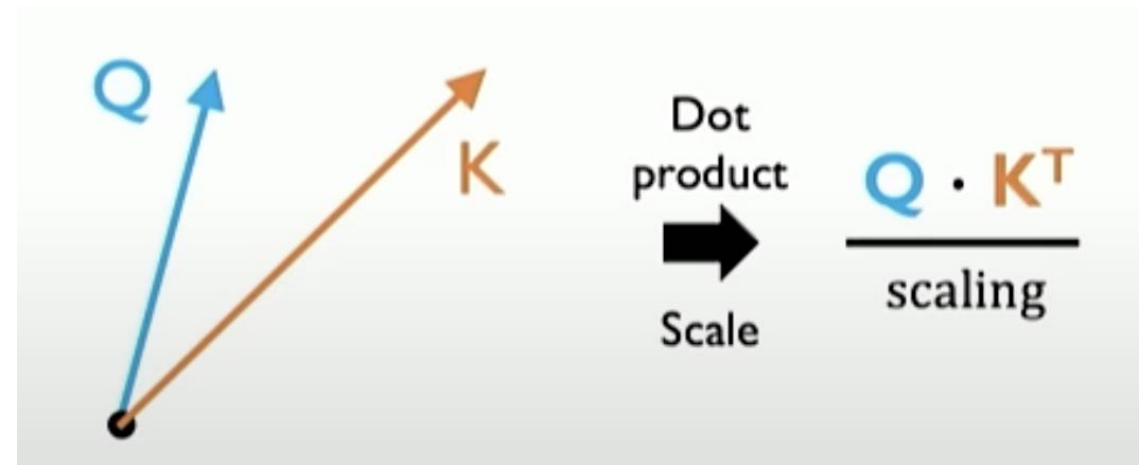
$$\text{softmax}\left(\frac{Q \cdot K^T}{\text{scaling}}\right)$$

Attention score: compute pairwise similarity between **query** and **key**

How to compute similarity between two sets of features?

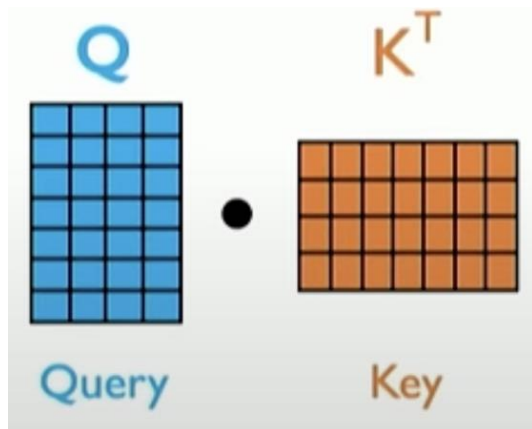
```
1 K.shape # (num_tokens, embedding_dim)
✓ 0.0s
torch.Size([6, 4])

1 Q.shape # (num_tokens, embedding_dim)
✓ 0.0s
torch.Size([6, 4])
```



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

3. Compute attention weights from Q and K



=

```
1 similarity_scores = Q @ K.T
2 similarity_scores.shape
✓ 0.0s
torch.Size([6, 6])
```

```
1 K.shape[1] #d_k
✓ 0.0s
4
```

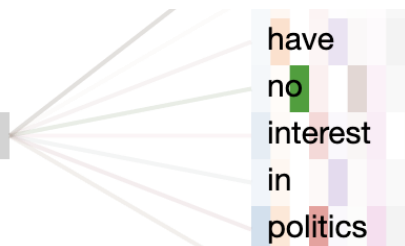
```
2 d_k = 4 # K.shape[1]
3 scaled_similarity = similarity_scores / torch.sqrt(torch.tensor(d_k))
4 scaled_similarity
✓ 0.0s
tensor([[ 0.8032,  1.2111,  1.1587,  0.7044,  0.2762,  0.2779],
        [-0.3347,  0.0165,  0.7382,  1.1743,  0.9310,  0.2396],
        [-0.9297, -0.6077,  0.2232,  0.7925,  0.5701, -0.2460],
        [-0.4320, -0.0807,  0.0933, -0.0886, -0.4782, -0.7360],
        [ 0.7036,  1.1197,  0.4743, -0.6574, -1.2530, -0.7831],
        [ 1.4358,  1.8970,  1.0222, -0.3836, -1.0454, -0.3476]])
```

(6, 4) · (4, 6) gives us a 6 x 6 matrix, where each row represents a word's similarity to every other word in the sentence.

$$\text{softmax}\left(\frac{Q \cdot K^T}{\text{scaling}}\right) =$$

```
1 attention_weights = torch.softmax(scaled_similarity, dim=1)
2 attention_weights
✓ 0.0s
tensor([[0.1662, 0.2499, 0.2371, 0.1505, 0.0981, 0.0983],
        [0.0658, 0.0935, 0.1925, 0.2977, 0.2334, 0.1169],
```

have
no
interest
in
politics



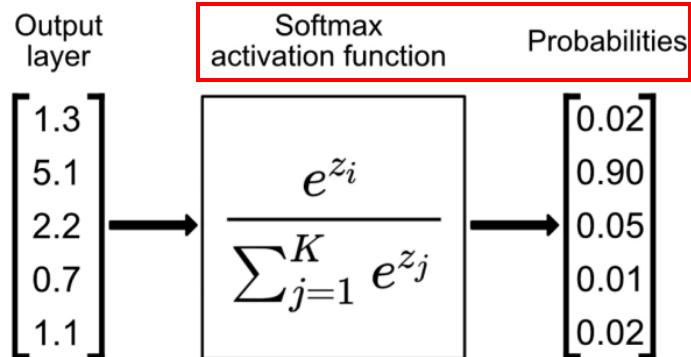
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Note: Remember the importance of softmax

$$\text{softmax}\left(\frac{Q \cdot K^T}{\text{scaling}}\right) =$$

```
1 attention_weights = torch.softmax(scaled_similarity, dim=1)
2 attention_weights
✓ 0.0s
tensor([[0.1662, 0.2499, 0.2371, 0.1505, 0.0981, 0.0983],
        [0.0658, 0.0935, 0.1925, 0.2977, 0.2334, 0.1169],
        [0.0568, 0.0784, 0.1799, 0.3179, 0.2545, 0.1125],
        [0.1386, 0.1969, 0.2344, 0.1954, 0.1324, 0.1023],
        [0.2541, 0.3853, 0.2021, 0.0652, 0.0359, 0.0575],
        [0.2732, 0.4333, 0.1807, 0.0440, 0.0229, 0.0459]])
```

Applying softmax to each row (dim = 1) of the 6x6 matrix gives us a probability distribution for each row



```
1 attention_weights[0]
✓ 0.0s
tensor([0.1662, 0.2499, 0.2371, 0.1505, 0.0981, 0.0983])
```

First row probabilities

Each row sums to 1

```
1 sum(attention_weights[0])
✓ 0.0s
tensor(1.0000)
```

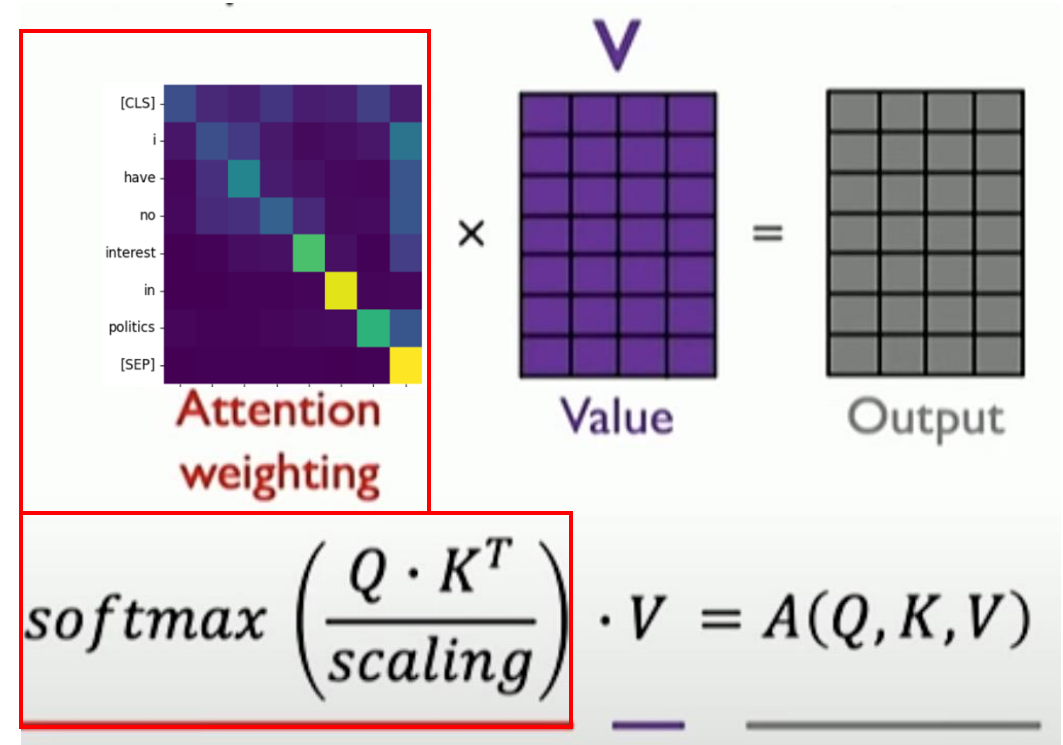
```
1 for row in attention_weights:
2     print(sum(row))
✓ 0.0s
tensor(1.0000)
tensor(1.0000)
tensor(1.0000)
tensor(1.)
tensor(1.0000)
tensor(1.0000)
```

4. Extract features with high attention

Goal: identify and attend to most important features in input

1. Encode **position** information
2. Extract **Q**, **K** and **V** for search
3. Compute **attention weighting**
4. Extract **features with high attention**

Self attend to extract features



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

4. Extract features with high attention



```
1 attention_weights = torch.softmax(scaled_similarity, dim=1)
2 attention_weights
```

✓ 0.0s

```
tensor([[0.1662, 0.2499, 0.2371, 0.1505, 0.0981, 0.0983],
        [0.0658, 0.0935, 0.1925, 0.2977, 0.2334, 0.1169],
        [0.0568, 0.0784, 0.1799, 0.3179, 0.2545, 0.1125],
        [0.1386, 0.1969, 0.2344, 0.1954, 0.1324, 0.1023],
        [0.2541, 0.3853, 0.2021, 0.0652, 0.0359, 0.0575],
        [0.2732, 0.4333, 0.1807, 0.0440, 0.0229, 0.0459]])
```

x

```
1 V = embeddings @ W_V
2 V
```

✓ 0.1s

```
tensor([[ -0.5408, -0.2027, -1.7963,  0.6842],
        [-0.1066,  1.5086, -1.8772, -0.3478],
        [-0.3294,  1.4019, -0.7125, -1.4968],
        [-1.0052, -0.4239,  0.6303, -1.6992],
        [-1.5132, -2.2891,  0.9197, -0.7614],
        [-1.3874, -2.4782, -0.1071,  0.4617]])
```

=

```
1 context = attention_weights @ V
2 context
```

✓ 0.0s

```
tensor([[ -0.6307,  0.1437, -0.7619, -0.6132],
        [-0.9237, -0.5527, -0.0412, -0.9053],
        [-0.9591, -0.6372,  0.0450, -0.9396],
        [-0.7118, -0.0417, -0.5518, -0.7101],
        [-0.4446,  0.5608, -1.2558, -0.3741],
        [-0.3960,  0.6669, -1.3891, -0.3052]])
```

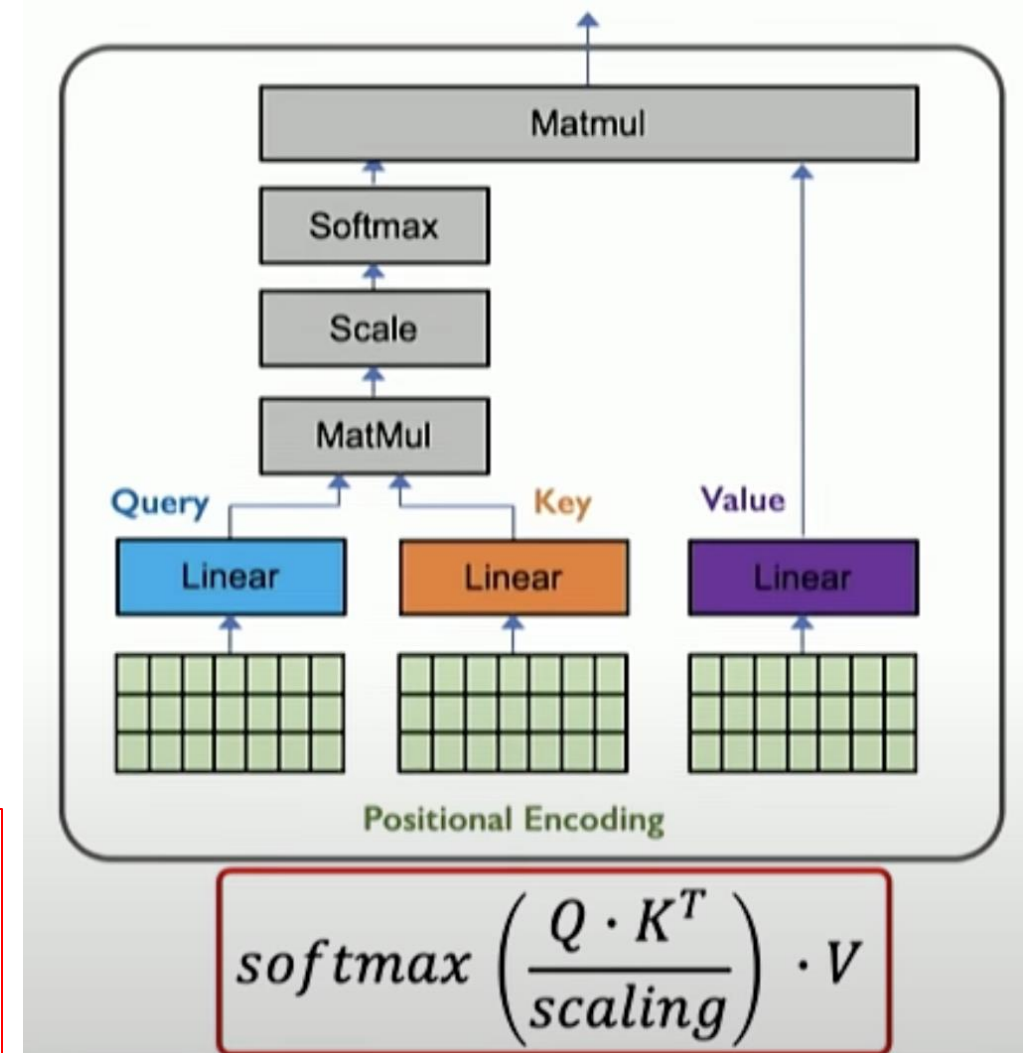
final retrieved document representation.

Self-Attention Head (transformations)

Goal: identify and attend to most important features in input

1. Encode **position** information
2. Extract **Q**, **K** and **V** for search
3. Compute **attention weighting**
4. Extract **features with high attention**

These features form a self-attention head
that can plug into a larger network
Each head attends to a different part of input



Applying Multiple Self-Attention Heads



Attention weighting

×



Value

=



Output



Output of attention head 1



Output of attention head 2

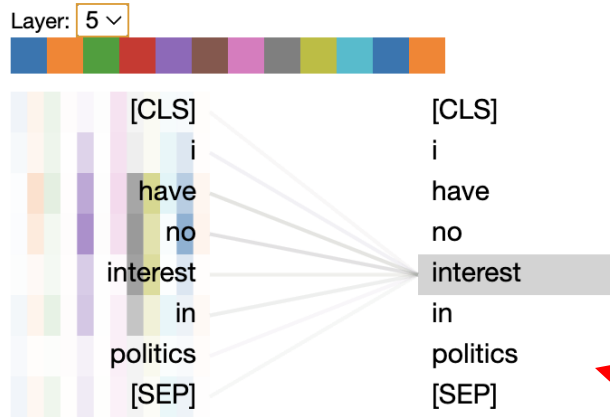


Output of attention head 3

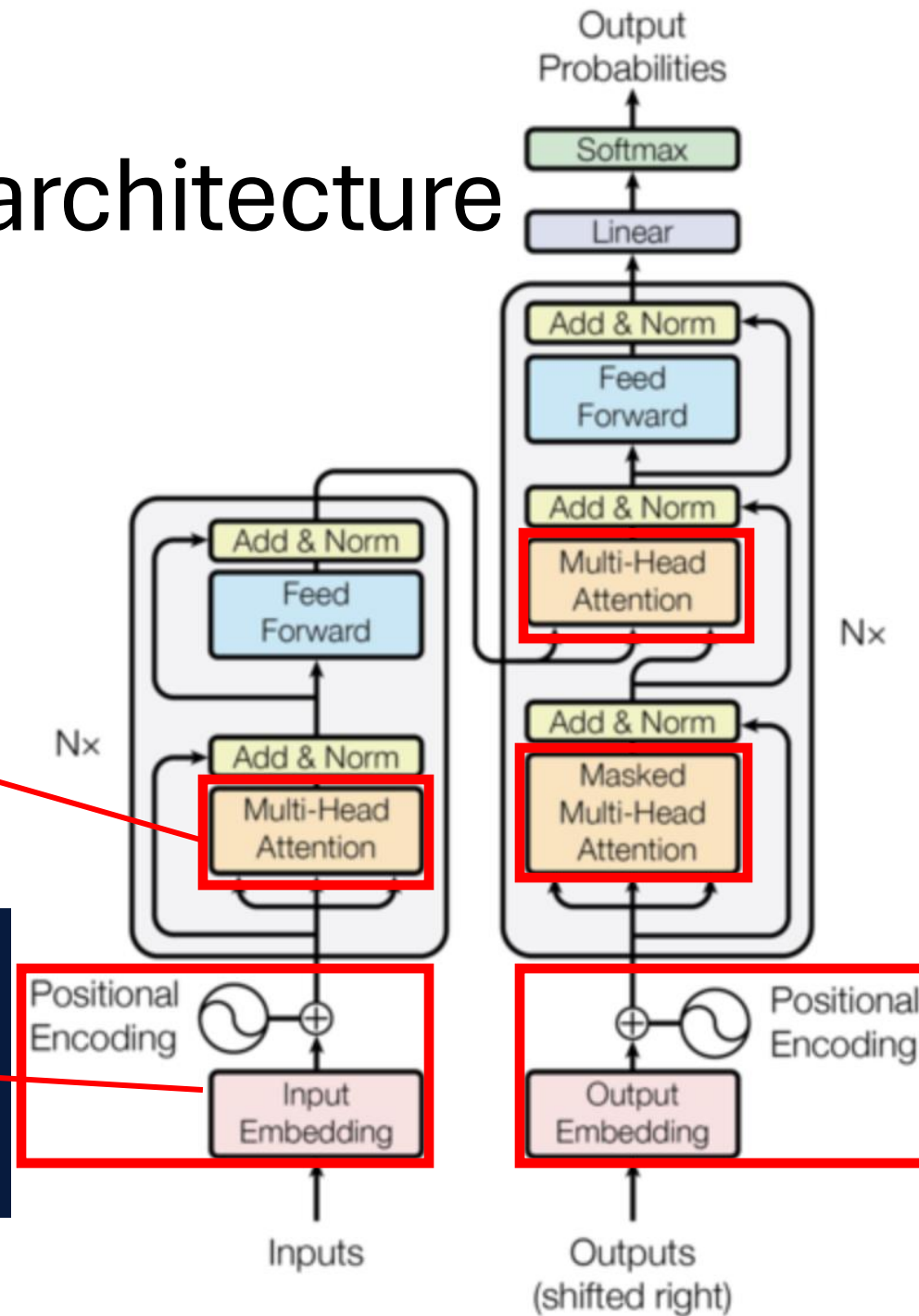
Transformation steps so far

- **Token embedding & positional encoding**
 - Transformation is the raw text -> dense vectors
- **Self-Attention** (Q, K, V, Softmax, Weighted Sum)
 - Transformation is the embeddings -> context—aware representations
- **Multi-head attention**
 - Transformation is the expansion of attention principle across multiple perspectives

Transformer architecture



```
embeddings = torch.tensor([
    [0.0000, 1.0000, 0.0000, 1.0000], # "I"
    [0.8415, 0.5403, 0.0100, 0.9999], # "have"
    [0.9093, -0.4161, 0.0200, 0.9998], # "no"
    [0.1411, -0.9900, 0.0300, 0.9996], # "interest"
    [-0.7568, -0.6536, 0.0400, 0.9992], # "in"
    [-0.9589, 0.2837, 0.0500, 0.9988] # "politics"
], dtype=torch.float32)
```

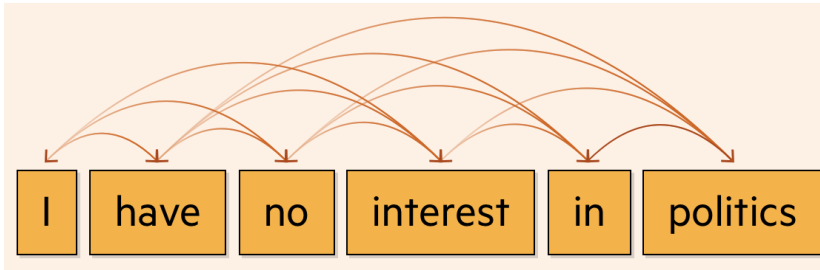
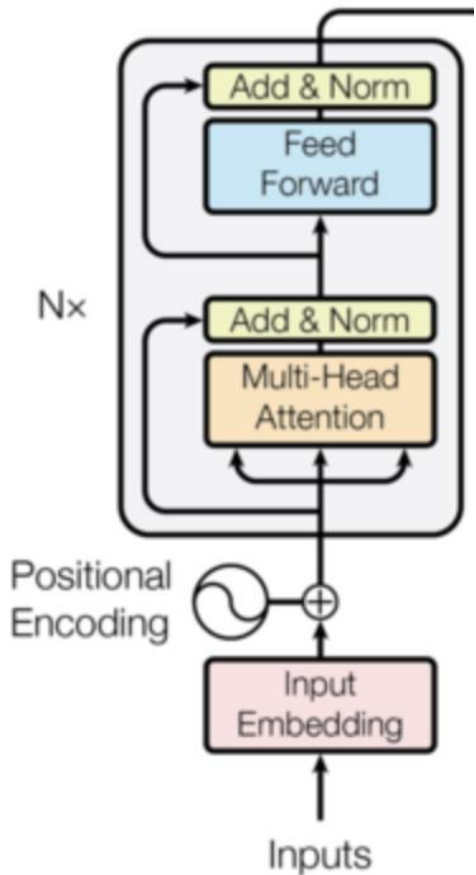


Transformation steps still to see

- **Feedforward Network (FFN)**
 - Transformation is the enhancement of learned features
- **Layer Normalisation & Residual Connections**
 - Transformation is the prevention of exploding/vanishing gradients
- **Decoder** (for **text generation** models like **GPT**)
 - Transformation is the conversion of **learned context** into **predictions**
- **Final Output Projection** (Logits & Softmax for probs.)
 - Transformation converts latent representations into **words!**

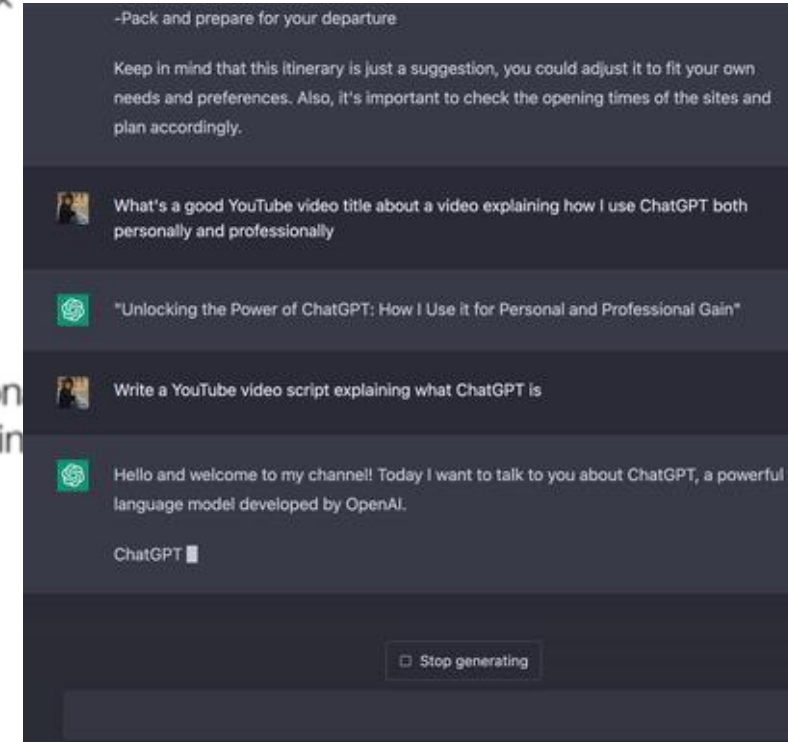
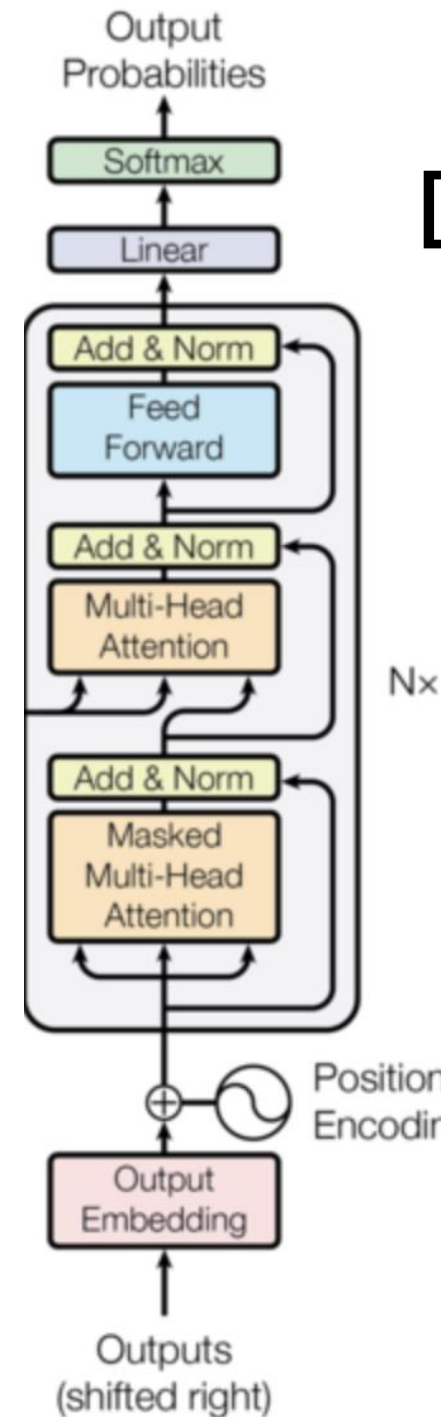
Encoder

Encoders (BERT) extract essential information from an input sequence and learn embeddings = text understanding

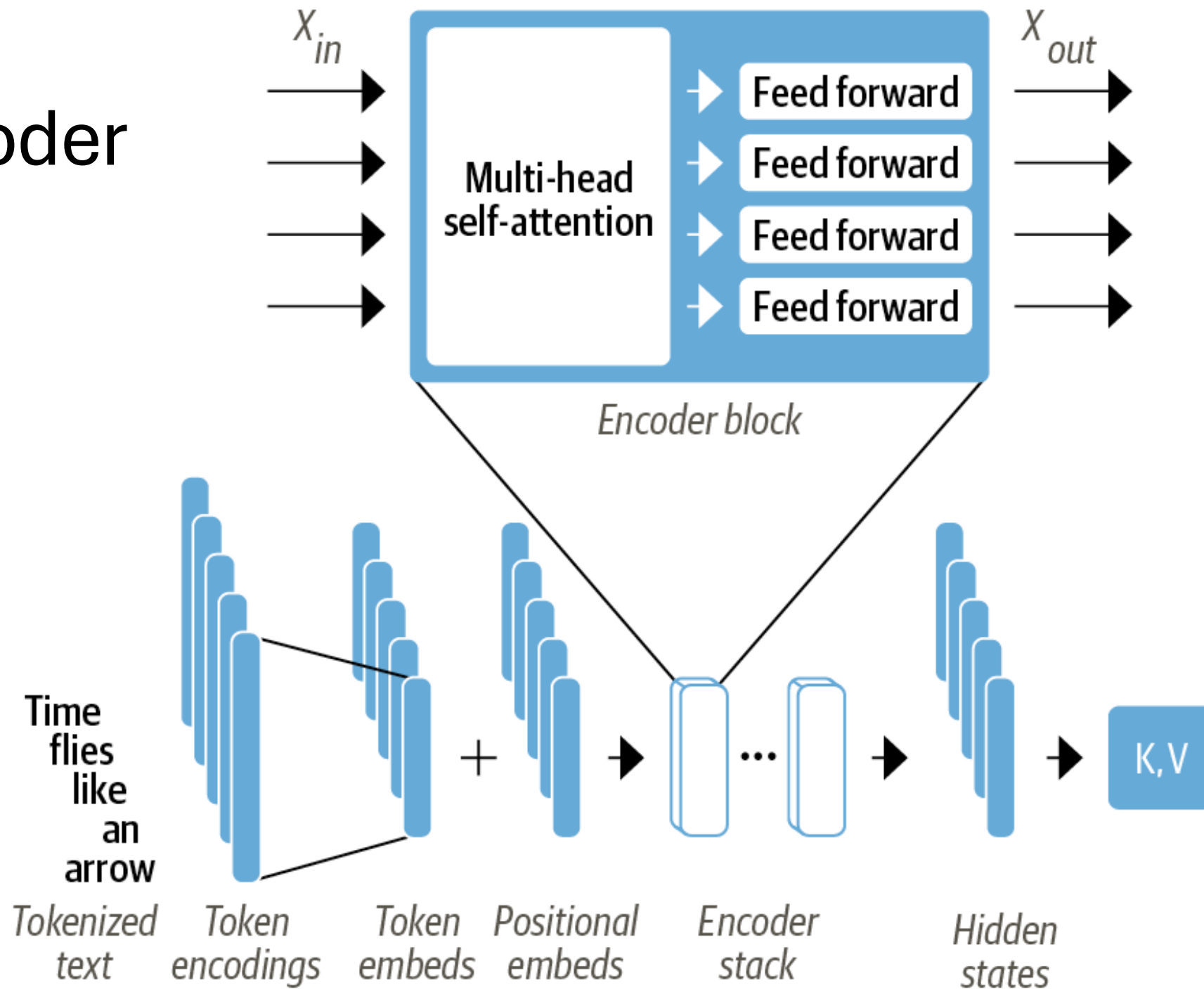


Decoder

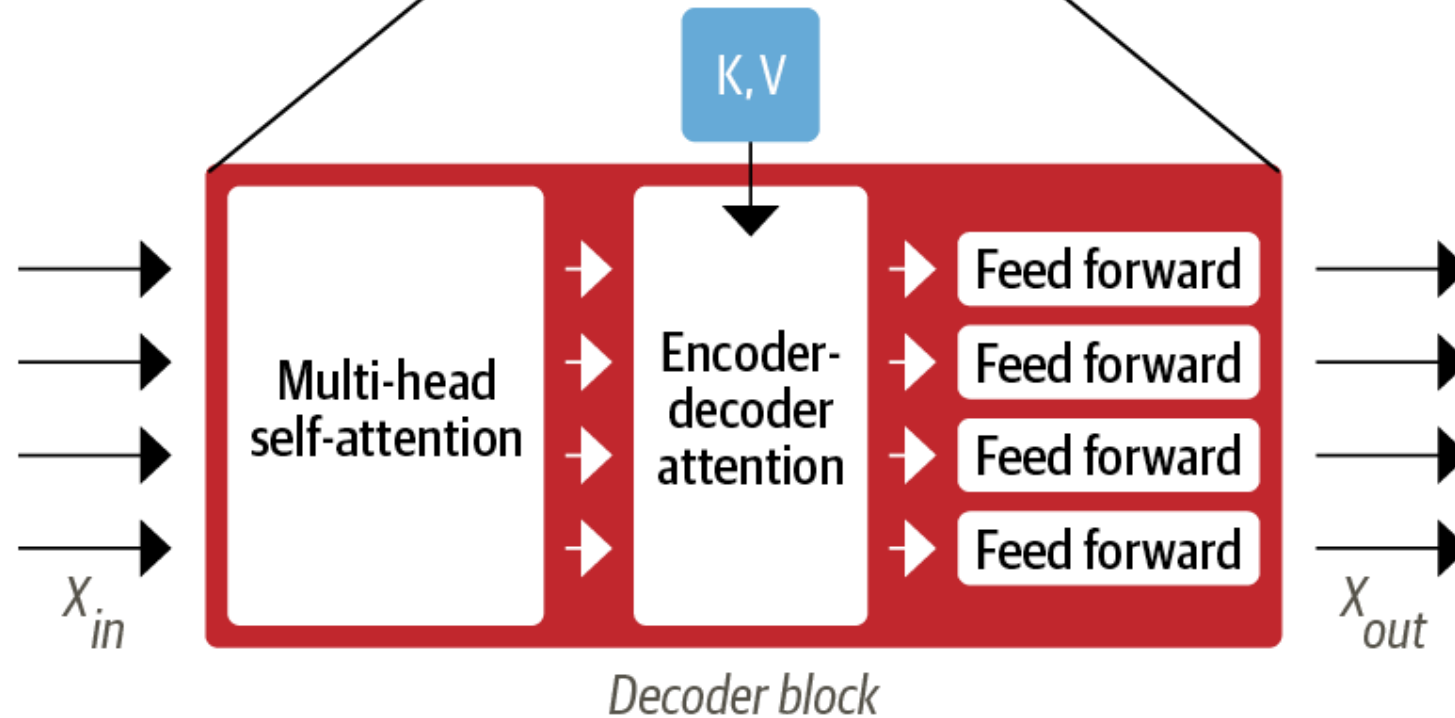
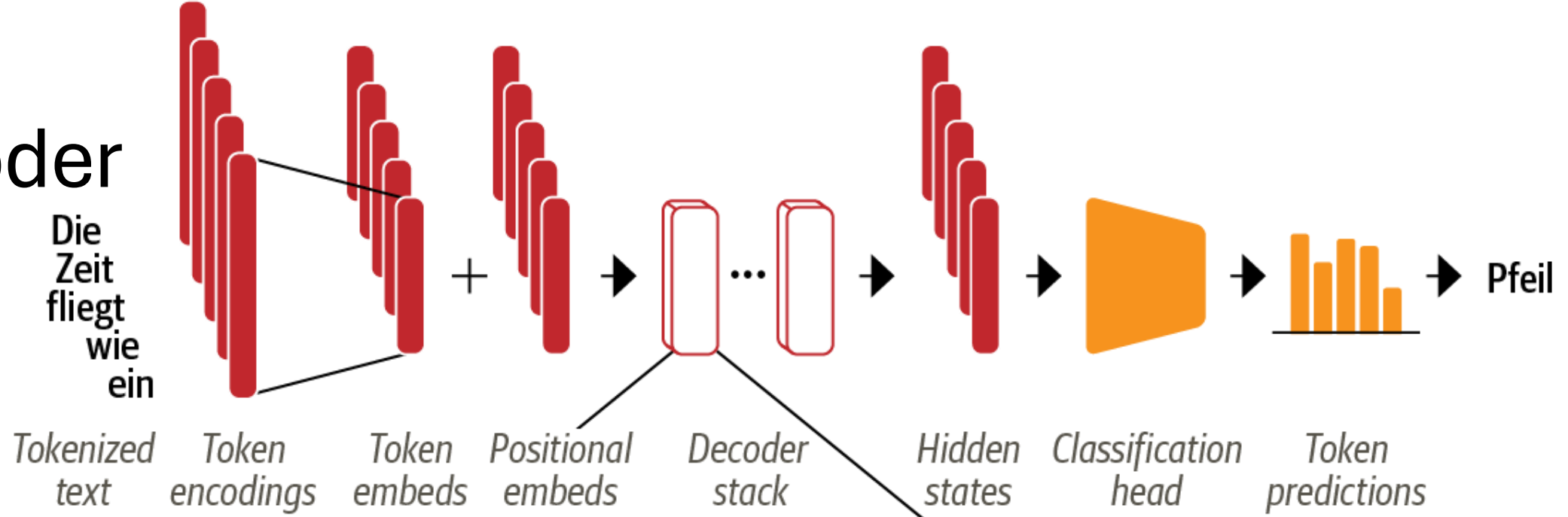
Decoders (GPT) are designed to predict new word sequences based on context provided (prompts) = text generation

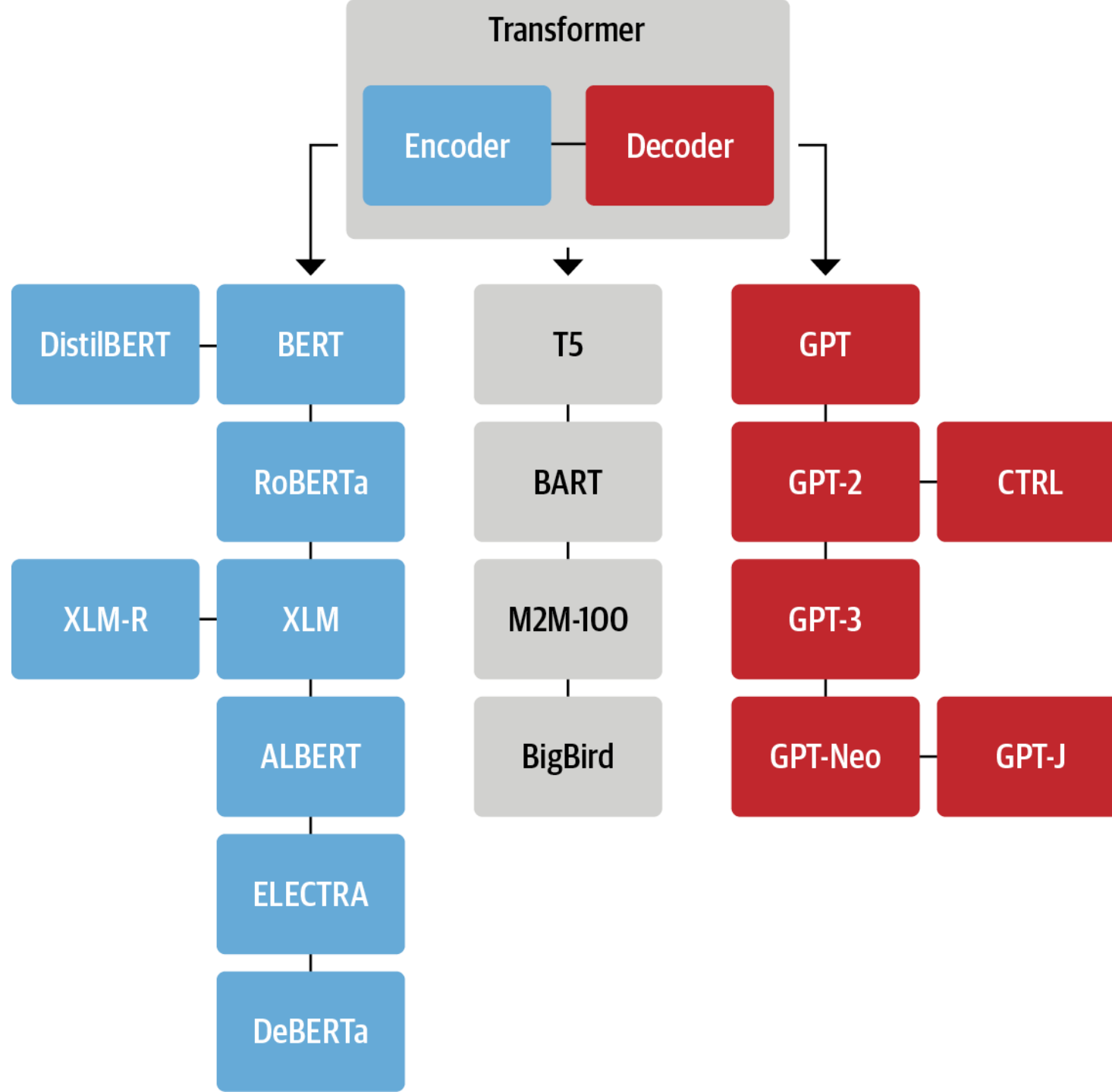


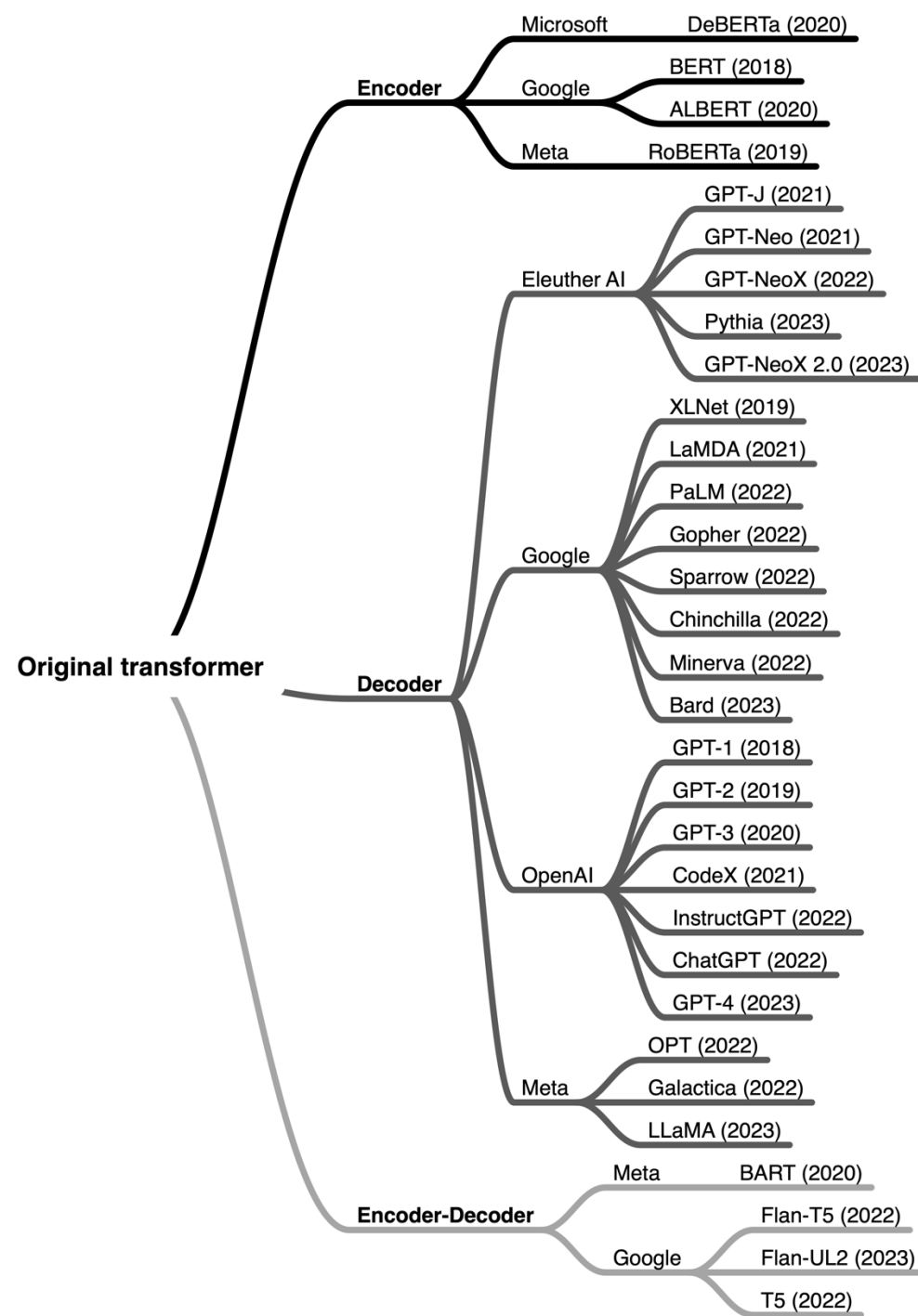
Encoder



Decoder







Encoder / Decoder – next lecture!

Encoder-Only Transformers (e.g., BERT)

- Use Case: Tasks requiring a holistic understanding of the input, such as classification, NER, and sentence similarity.
- Stack of encoder layers (multi-head self-attention + FNN).
- Processes bidirectional context.
- No decoder.

Decoder-Only Transformers (e.g., GPT)

- Use Case: Autoregressive tasks like text generation.
- Stack of decoder layers (masked multi-head self-attention + FNN).
- Masked attention ensures tokens only attend to earlier tokens in the sequence, preserving causality.
- No encoder.

Further Reading

Natural Language Processing with Transformers by Tunstall et al. 2022

Notebooks available at:

<https://github.com/nlp-with-transformers/notebooks/tree/main>

